

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn bất động sản tích hợp Chatbot

TRẦN TRUNG HIẾU

Email: hieu.tt227114@sis.hust.edu.vn

Mã sinh viên: 20227114

Chuyên ngành Toán - Tin

Giảng viên hướng dẫn:

TS. Trần Anh Tú

Chữ ký của GVHD

HÀ NỘI, 6/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn bất động sản tích hợp Chatbot

TRẦN TRUNG HIẾU

Email: hieu.tt227114@sis.hust.edu.vn

Mã sinh viên: 20227114

Chuyên ngành Toán - Tin

Giảng viên hướng dẫn:

TS. Trần Anh Tú

Chữ ký của GVHD

HÀ NỘI, 6/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Biểu mẫu của Đề tài/khóa luận tốt nghiệp theo qui định của viện, tuy nhiên cần đảm bảo giáo viên giao đề tài ký và ghi rõ họ và tên.

Trường hợp có 2 giáo viên hướng dẫn thì sẽ cùng ký tên.

Hà Nội, ngày tháng ...
năm ...

Giáo viên hướng dẫn

Ký và ghi rõ họ tên

PHIẾU BÁO CÁO TIẾN ĐỘ ĐỒ ÁN

- Phiếu này in từ hệ thống số hóa

Lời cảm ơn

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến TS. Trần Anh Tú, người đã tận tình hướng dẫn, định hướng và động viên em trong suốt quá trình thực hiện đồ án tốt nghiệp này. Những góp ý quý báu của thầy về kiến trúc hệ thống, phương pháp nghiên cứu và cách trình bày đã giúp em hoàn thiện đồ án một cách tốt nhất.

Em cũng xin gửi lời cảm ơn đến các thầy cô trong Khoa Toán – Tin, Đại học Bách Khoa Hà Nội, những người đã trang bị cho em nền tảng kiến thức vững chắc về công nghệ thông tin, trí tuệ nhân tạo và kỹ năng nghiên cứu trong suốt những năm học vừa qua.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, tạo điều kiện thuận lợi và là nguồn động viên tinh thần to lớn để em có thể tập trung hoàn thành đồ án này.

Tóm tắt nội dung đồ án

Đồ án này trình bày quá trình phân tích, thiết kế và xây dựng một nền tảng tư vấn bất động sản thông minh tích hợp chatbot đa tác tử (multi-agent) sử dụng kỹ thuật Retrieval-Augmented Generation (RAG). Hệ thống hướng đến thị trường bất động sản Việt Nam, cung cấp cho người dùng khả năng tìm kiếm nhà đất, phân tích thị trường, tư vấn pháp lý và đầu tư thông qua giao diện hội thoại tự nhiên bằng tiếng Việt.

Về phương pháp thực hiện, đồ án áp dụng kiến trúc microservices với backend FastAPI, frontend Next.js, cơ sở dữ liệu PostgreSQL tích hợp pgvector cho tìm kiếm ngữ nghĩa, và framework LangGraph để điều phối đa tác tử. Hệ thống sử dụng mô hình nhúng BGE-M3 (1024 chiều) cho embedding tiếng Việt, Google Gemini 2.0 Flash cho phân loại ý định và sinh phản hồi, cùng Cohere Rerank để xếp hạng kết quả truy xuất. Dữ liệu được thu thập từ batdongsan.com.vn qua pipeline ETL tự động hóa bằng Apache Airflow.

Kết quả đạt được là một hệ thống hoàn chỉnh với các chức năng: duyệt và lọc danh sách bất động sản, xem chi tiết và bản đồ, thống kê thị trường, và đặc biệt là chatbot đa tác tử có khả năng xử lý đồng thời nhiều loại truy vấn (tìm kiếm, phân tích thị trường, tư vấn pháp lý, tư vấn đầu tư) với nguồn trích dẫn rõ ràng. Hệ thống có tính thực tế cao, có thể triển khai làm nền tảng dịch vụ bất động sản hoặc phát triển mở rộng theo hướng tích hợp thêm nguồn dữ liệu, cải thiện chất lượng embedding và bổ sung agent chuyên biệt.

Qua đồ án, em đã đạt được kiến thức và kỹ năng về thiết kế hệ thống full-stack, xây dựng pipeline dữ liệu, tích hợp vector database cho tìm kiếm ngữ nghĩa, phát triển hệ thống multi-agent với LangGraph, và triển khai ứng dụng với Docker.

Hà Nội, ngày tháng ...
năm...

Sinh viên thực hiện

Ký và ghi rõ họ tên

Mục lục

1	GIỚI THIỆU	11
1.1	Đặt vấn đề	11
1.2	Mục tiêu của đề tài	12
1.3	Phạm vi nghiên cứu	14
1.4	Phương pháp tiếp cận	15
1.5	Đóng góp của đồ án	16
1.6	Bố cục của báo cáo	16
2	CƠ SỞ LÝ THUYẾT	18
2.1	Tổng quan về lĩnh vực nghiên cứu	18
2.1.1	Nền tảng bất động sản trực tuyến	18
2.1.2	Chatbot và trợ lý ảo trong bất động sản	19
2.2	Các khái niệm cơ bản	19
2.2.1	Retrieval-Augmented Generation (RAG)	19
2.2.2	Multi-Agent System (Hệ đa tác tử)	20
2.2.3	Vector Embedding và Tìm kiếm ngữ nghĩa	23
2.2.4	Hybrid Search và Reranking	24
2.3	Các công nghệ sử dụng	25
2.3.1	FastAPI	25
2.3.2	Next.js	26
2.3.3	PostgreSQL + pgvector	26
2.3.4	LangGraph	27
2.3.5	Google Gemini	27
2.3.6	BGE-M3 Embedding	28
2.3.7	Cohere Rerank	28
2.3.8	Redis	29
2.3.9	Apache Airflow	29
2.3.10	Playwright	30
2.3.11	Các thư viện và công cụ khác	30
3	PHÂN TÍCH THIẾT KẾ HỆ THỐNG	32
3.1	Phân tích yêu cầu	32
3.1.1	Yêu cầu chức năng	32

3.1.2	Yêu cầu phi chức năng	37
3.2	Thiết kế kiến trúc hệ thống	39
3.2.1	Kiến trúc tổng thể	39
3.3	Thiết kế cơ sở dữ liệu	46
3.3.1	Mô hình quan hệ	46
3.3.2	Chiến lược indexing	48
3.4	Thiết kế giao diện người dùng	48
3.4.1	Trang chủ (Home Page)	48
3.4.2	Trang danh sách bất động sản (Listing Pages)	48
3.4.3	Trang chi tiết bất động sản (Detail Page)	49
3.4.4	Trang thống kê thị trường (Market Dashboard)	49
3.4.5	Trang quản trị (Admin Dashboard)	49
3.4.6	Chat Widget	50
4	TRIỂN KHAI VÀ KIỂM THỬ	51
4.1	Triển khai hệ thống với Docker	51
4.1.1	Kiến trúc triển khai	51
4.1.2	Cấu hình Docker Compose	51
4.1.3	Cấu hình chi tiết từng service	52
4.1.4	Mạng nội bộ và giao tiếp giữa các service	55
4.1.5	Quy trình triển khai	56
4.1.6	Nạp dữ liệu ban đầu	58
4.2	Kiểm thử hệ thống	59
4.2.1	Chiến lược kiểm thử	59
4.2.2	Công cụ và framework kiểm thử	59
	KẾT LUẬN	61
	Kết quả đạt được	61
	Hạn chế	62
	PHỤ LỤC	63
	CHỈ MỤC	75
	TÀI LIỆU THAM KHẢO	76

Danh sách hình vẽ

3.1	Kiến trúc tổng thể hệ thống Real Estate Chatbot Platform	40
4.1	Kiến trúc triển khai Docker Compose	51
4.2	Sơ đồ kiến trúc tổng thể hệ thống (5 tầng: Người dùng – Frontend – Backend & Agent Service – Data – Pipeline)	68

Danh sách bảng

3.1	Các yêu cầu phi chức năng của hệ thống	39
4.1	Danh sách API endpoints chính của hệ thống (tiền tố mặc định: <code>/api/v1</code>)	72

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

Ký hiệu / Từ viết tắt	Diễn giải
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
BGE-M3	BAAI General Embedding – Multilingual, Multi-functionality, Multi-granularity
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
ETL	Extract, Transform, Load
HNSW	Hierarchical Navigable Small World (thuật toán index vector)
JWT	JSON Web Token
kNN	k-Nearest Neighbors
LLM	Large Language Model – Mô hình ngôn ngữ lớn
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
ROI	Return on Investment
SQL	Structured Query Language
SSE	Server-Sent Events
SSR	Server-Side Rendering
VND	Việt Nam Đồng

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Thị trường bất động sản Việt Nam những năm gần đây chứng kiến sự phát triển sôi động với hàng trăm nghìn giao dịch mỗi năm và nhu cầu thông tin ngày càng cao từ phía người mua, người bán cũng như nhà đầu tư. Các cổng thông tin bất động sản trực tuyến như batdongsan.com.vn, alonhadat.com.vn đã trở thành kênh tiếp cận chính, với hàng chục nghìn tin đăng mới mỗi ngày. Tuy nhiên, việc khai thác thông tin từ các nền tảng này một cách hiệu quả vẫn còn nhiều thách thức:

1. **Thông tin phân mảnh và quá tải:** Dữ liệu bất động sản nằm rải rác trên nhiều nền tảng với hàng chục nghìn tin đăng. Người dùng phải tự duyệt qua hàng trăm kết quả, so sánh thủ công giữa các tin để tìm được bất động sản phù hợp. Việc tổng hợp thông tin từ nhiều nguồn và đưa ra quyết định dựa trên dữ liệu là một quá trình tốn nhiều thời gian và công sức.
2. **Thiếu tư vấn chuyên sâu đa lĩnh vực:** Một giao dịch bất động sản không chỉ đơn thuần là tìm kiếm và so sánh giá. Người mua cần được tư vấn về tình trạng pháp lý (sổ đỏ, quy hoạch, thủ tục sang tên), xu hướng thị trường (biến động giá, khu vực tiềm năng) và tiềm năng đầu tư (khả năng sinh lời, thanh khoản, tỷ suất cho thuê). Tuy nhiên, các nền tảng hiện tại chủ yếu chỉ hiển thị danh sách tin đăng và bộ lọc cơ bản, chưa có khả năng tư vấn tổng hợp đa chiều.
3. **Rào cản ngôn ngữ trong tìm kiếm ngữ nghĩa:** Các giải pháp tìm kiếm truyền thống dựa trên từ khóa chính xác (exact keyword matching) không hiểu được ngữ nghĩa và ngữ cảnh của truy vấn tiếng Việt. Ví dụ, người dùng tìm “căn hộ thoáng mát gần công viên, dưới 3 tỷ” yêu cầu hệ thống phải hiểu được khái niệm “thoáng mát”, “gần công viên” – những khái niệm không thể tra cứu bằng SQL thông thường.
4. **Thiếu tự động hóa trong thu thập và xử lý dữ liệu:** Quy trình thu thập dữ liệu bất động sản thường thủ công, không theo kịp tốc độ cập nhật của thị trường. Dữ liệu thô từ các nền tảng cần được làm sạch (chuẩn hóa giá, diện tích, địa chỉ), làm giàu (gán tọa độ địa lý, phân loại ý định) và cập nhật định kỳ mới có thể sử dụng được cho các hệ thống tìm kiếm thông minh.
5. **Thiếu công cụ phân tích thị trường trực quan:** Người dùng phổ thông và nhà đầu tư thiếu các công cụ giúp họ nắm bắt bức tranh tổng quan về thị trường – giá trung

bình theo khu vực, phân bố loại hình bất động sản, xu hướng biến động giá. Các số liệu thống kê thường nằm trong các báo cáo định kỳ, không có sẵn dưới dạng tương tác trực tuyến.

Trước bối cảnh đó, việc xây dựng một nền tảng bất động sản thông minh, tích hợp trợ lý ảo có khả năng hiểu ngôn ngữ tự nhiên tiếng Việt và tư vấn đa lĩnh vực – từ tìm kiếm bất động sản, phân tích thị trường, tư vấn pháp lý đến đánh giá đầu tư – là một nhu cầu cấp thiết. Hệ thống cần được xây dựng trên nền tảng dữ liệu được thu thập và cập nhật tự động, đảm bảo thông tin luôn mới và đáng tin cậy.

1.2. Mục tiêu của đề tài

Đề án hướng đến việc xây dựng một hệ thống toàn diện giải quyết các thách thức nêu trên, với 5 mục tiêu chính:

1. Xây dựng nền tảng web bất động sản hoàn chỉnh:

- Giao diện người dùng hiện đại với Next.js 16 + React 19, hỗ trợ SSR (Server-Side Rendering) cho SEO và hiệu năng cao.
- Trang duyệt danh sách nhà đất bán và cho thuê với phân trang và sắp xếp đa tiêu chí (giá, diện tích, ngày đăng).
- Bộ lọc nâng cao: loại tin, loại hình bất động sản, thành phố, quận/huyện, khoảng giá, diện tích, số phòng ngủ, số phòng tắm, hướng nhà, tìm kiếm theo từ khóa.
- Trang chi tiết bất động sản với đầy đủ thông số kỹ thuật (giá, diện tích, pháp lý, nội thất, hướng, mặt tiền, số tầng, tọa độ).
- Dashboard thống kê thị trường trực quan với các chỉ số tổng quan và biểu đồ phân tích.
- Hệ thống xác thực người dùng qua JWT (đăng ký, đăng nhập, phiên đăng nhập 24 giờ).
- Bảng điều khiển quản trị (admin dashboard) theo dõi trạng thái pipeline và agent traces.

2. Phát triển chatbot đa tác tử (Multi-Agent) tích hợp RAG:

- Pipeline chatbot nội bộ trong backend với router phân loại ý định (keyword-based và Gemini-based) và 4 agent chuyên biệt: Property Search, Market Analysis, Legal Advisor, Investment Advisor.

- Agent Service riêng biệt chạy LangGraph StateGraph với quy trình 8 bước: context builder → readiness checker → router → retrieval planner → specialist agents (chạy song song) → synthesizer → safety validator → memory proposals.
- Hỗ trợ tiếng Việt toàn diện – từ nhận diện ý định, trích xuất bộ lọc đến sinh phản hồi.
- Cơ chế ghi nhớ người dùng (memory) qua UserPreference và MemoryProposal, cho phép chatbot cá nhân hóa phản hồi dựa trên lịch sử tương tác.
- Cơ chế đánh giá chất lượng (LLM judge) và thu thập phản hồi người dùng (chat feedback).

3. Xây dựng pipeline thu thập và xử lý dữ liệu tự động:

- Crawler sử dụng Playwright với cơ chế stealth để thu thập dữ liệu từ batdongsan.com.vn cho 4 loại nội dung: tin bán, tin cho thuê, dự án, tin tức.
- Pipeline ETL đầy đủ: làm sạch (parse giá, diện tích, phân loại) → làm giàu (geocoding qua Nominatim/Goong, intent tagging) → chunking ngữ nghĩa → tạo embedding BGE-M3 (1024 chiều) → nạp vào PostgreSQL + pgvector qua các ingestor module.
- Hệ thống kiến thức pháp lý (Legal KB) với khả năng xử lý tài liệu HTML và PDF.
- Lập lịch tự động qua Apache Airflow với 4 DAGs: hàng ngày (tin đăng), hàng tuần (dự án, tin tức), hàng tháng (kiến thức pháp lý).

4. Triển khai hệ thống tìm kiếm lai (Hybrid Search) hiệu quả:

- Kết hợp tìm kiếm theo bộ lọc có cấu trúc (SQL WHERE) với tìm kiếm ngữ nghĩa qua pgvector HNSW index.
- Xếp hạng lại (reranking) qua Cohere Rerank Multilingual v3.0 để cải thiện độ chính xác.
- Redis caching cho cả embedding và kết quả tìm kiếm, giảm độ trễ cho truy vấn lặp lại.
- Hỗ trợ trích xuất bộ lọc tự động từ truy vấn tiếng Việt (giá, diện tích, khu vực, loại hình).

5. Thiết kế kiến trúc microservices có khả năng mở rộng:

- 5 service độc lập trong Docker Compose: PostgreSQL + pgvector (cơ sở dữ liệu), Redis (cache), Backend API (FastAPI), Agent Service (FastAPI + LangGraph), Frontend (Next.js).
- Hệ thống observability toàn diện: AgentTrace, AgentTraceStep, AgentLLMCall, AgentRetrievalEvent cho tracing; Prometheus metrics cho giám sát; ChatFeedback cho đánh giá chất lượng.
- Bộ test hơn 55 file bao phủ toàn bộ các module: models, API, agent service, RAG, data pipeline, legal KB, crawler, Airflow DAGs.

1.3. Phạm vi nghiên cứu

Đề án tập trung vào các phạm vi sau:

- **Thị trường:** Bất động sản Việt Nam. Dữ liệu được thu thập từ nguồn chính là batdongsan.com.vn – cổng thông tin bất động sản lớn nhất Việt Nam với hàng chục nghìn tin đăng mới mỗi ngày.
- **Loại hình bất động sản:** Bao gồm cả tin bán và cho thuê với các loại hình chính: căn hộ chung cư, nhà riêng, nhà mặt phố, đất nền, biệt thự, nhà phố thương mại (shophouse).
- **Chức năng chính:**
 - *Người dùng cuối:* Tìm kiếm và duyệt bất động sản với bộ lọc nâng cao; chatbot tư vấn đa lĩnh vực (tìm kiếm, thị trường, pháp lý, đầu tư); thống kê thị trường trực quan; xác thực và quản lý tài khoản.
 - *Quản trị viên:* Giám sát pipeline dữ liệu qua admin dashboard; theo dõi agent traces và hiệu năng hệ thống; quản lý phản hồi người dùng và memory proposals.
 - *Hệ thống:* Thu thập dữ liệu tự động theo lịch; xử lý ETL toàn diện; tạo và lập chỉ mục embedding; giám sát qua Prometheus metrics.
- **Công nghệ sử dụng:**
 - **Frontend:** Next.js 16 + React 19 (TypeScript), App Router, Tailwind CSS v4.
 - **Backend API:** FastAPI (Python 3.12), SQLAlchemy 2.0 async, Pydantic v2.
 - **Agent Service:** FastAPI + LangGraph với StateGraph 8-node workflow, Google Gemini 2.0 Flash.
 - **Cơ sở dữ liệu:** PostgreSQL 16 + pgvector (HNSW index, cosine distance).

- **Cache:** Redis 7 (caching embedding, kết quả tìm kiếm, session).
 - **AI/ML:** Google Gemini 2.0 Flash (LLM + router), BGE-M3 BAAI/bge-m3 (embedding 1024 chiều), Cohere Rerank Multilingual v3.0 (xếp hạng lại).
 - **Data Pipeline:** Apache Airflow (lập lịch), Playwright (crawler), Nominatim/Goong (geocoding).
 - **Giám sát:** Prometheus metrics, AgentTrace observability system.
- **Ngôn ngữ:**
 - Giao diện người dùng và phản hồi chatbot: tiếng Việt.
 - Mã nguồn, docstring, tên biến/hàm/lớp: tiếng Anh.
 - URL slugs: tiếng Việt không dấu (/nha-dat-ban, /thi-truong, /dang-nhap).

1.4. Phương pháp tiếp cận

Đồ án được triển khai theo phương pháp phát triển module, kết hợp giữa xây dựng hạ tầng dữ liệu và phát triển ứng dụng:

1. **Thu thập và xử lý dữ liệu:** Bước đầu tiên là xây dựng pipeline thu thập dữ liệu tự động từ batdongsan.com.vn. Dữ liệu thô được làm sạch, chuẩn hóa, làm giàu (geocoding, intent tagging) và tổ chức thành các bảng quan hệ trong PostgreSQL. Văn bản được chia thành các đoạn ngữ nghĩa (chunks) và tạo vector embedding bằng mô hình BGE-M3 để phục vụ tìm kiếm ngữ nghĩa.
2. **Xây dựng backend API:** Backend được phát triển với FastAPI theo kiến trúc phân lớp rõ ràng: models (ORM) → schemas (Pydantic validation) → routers (API endpoints) → services (business logic). Các API cung cấp đầy đủ chức năng cho frontend: truy vấn danh sách bất động sản với bộ lọc và sắp xếp, thống kê thị trường, xác thực người dùng, và quan trọng nhất – tích hợp chatbot multi-agent.
3. **Phát triển chatbot đa tác tử:** Chatbot được triển khai theo kiến trúc hai lớp:
 - *Lớp 1 – Pipeline nội bộ:* Được tích hợp trực tiếp trong backend, sử dụng router keyword-based (có fallback Gemini) để phân loại ý định và orchestrator gọi song song các agent chuyên biệt. Đây là phương thức xử lý chính, đảm bảo độ trễ thấp và hoạt động ổn định ngay cả khi không có API key Gemini.
 - *Lớp 2 – Agent Service:* Microservice riêng biệt sử dụng LangGraph StateGraph với quy trình 8 bước, hỗ trợ multi-step reasoning, retrieval planning và LLM

judge đánh giá chất lượng. Service này được kích hoạt tùy chọn cho các truy vấn phức tạp.

4. **Xây dựng giao diện người dùng:** Frontend được phát triển với Next.js App Router, tập trung vào trải nghiệm người dùng mượt mà và hiệu năng cao. Giao diện được tổ chức thành các thành phần (components) tái sử dụng, với chatbot widget nổi cho phép người dùng tương tác mọi lúc.
5. **Kiểm thử và đảm bảo chất lượng:** Hệ thống được kiểm thử với hơn 55 file test bao phủ tất cả các module. Các bài test bao gồm unit test (models, schemas, utilities), integration test (API endpoints, database operations) và end-to-end test (chatbot pipeline, data ingestion).

1.5. Đóng góp của đề án

Đề án có những đóng góp chính sau:

1. **Về mặt ứng dụng:** Xây dựng một nền tảng bất động sản hoàn chỉnh, sẵn sàng triển khai thực tế, tích hợp công nghệ AI/ML tiên tiến (RAG, multi-agent, vector search) để giải quyết bài toán tư vấn bất động sản đa lĩnh vực bằng tiếng Việt.
2. **Về mặt kiến trúc:** Đề xuất kiến trúc chatbot hai lớp (pipeline nội bộ + agent service) với cơ chế fallback linh hoạt, đảm bảo hệ thống hoạt động ổn định trong nhiều điều kiện vận hành khác nhau (có/không có API key, có/không có agent service).
3. **Về mặt dữ liệu:** Xây dựng pipeline thu thập và xử lý dữ liệu bất động sản tự động, hoàn chỉnh từ khâu crawl đến embedding, có khả năng vận hành định kỳ qua Apache Airflow. Hệ thống legal KB cung cấp khả năng xử lý và truy xuất kiến thức pháp lý từ nhiều định dạng tài liệu.
4. **Về mặt kỹ thuật:** Tích hợp và đánh giá hiệu quả của hybrid search (SQL + pgvector HNSW + Cohere rerank + Redis cache) cho bài toán tìm kiếm bất động sản tiếng Việt, với khả năng trích xuất bộ lọc tự động từ truy vấn ngôn ngữ tự nhiên.
5. **Về mặt vận hành:** Thiết lập hệ thống observability toàn diện (AgentTrace, Prometheus metrics, ChatFeedback) và bộ test phong phú, tạo nền tảng vững chắc cho việc phát triển và vận hành hệ thống trong môi trường production.

1.6. Bố cục của báo cáo

Báo cáo được tổ chức thành 4 chương chính và phần kết luận:

- **Chương 1 – Giới thiệu:** Đặt vấn đề về những thách thức trong tiếp cận thông tin bất động sản tại Việt Nam; xác định 5 mục tiêu chính của đề án; phạm vi nghiên cứu về thị trường, loại hình, chức năng và công nghệ; phương pháp tiếp cận theo hướng phát triển module; và những đóng góp chính của đề án.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các khái niệm nền tảng: Retrieval-Augmented Generation (RAG), hệ đa tác tử (Multi-Agent System), vector embedding và tìm kiếm ngữ nghĩa, hybrid search và reranking. Giới thiệu các công nghệ được sử dụng: FastAPI, Next.js, PostgreSQL + pgvector, LangGraph, Google Gemini, BGE-M3 Embedding, Cohere Rerank, Redis, Apache Airflow và Playwright.
- **Chương 3 – Phân tích thiết kế hệ thống:** Phân tích yêu cầu chức năng (3 nhóm: người dùng cuối, chatbot, quản trị/dữ liệu) và phi chức năng. Thiết kế kiến trúc tổng thể 5 tầng với sơ đồ kiến trúc; thiết kế chi tiết các tầng (frontend, backend API, agent service, data layer, data pipeline); thiết kế cơ sở dữ liệu với 4 nhóm bảng và chiến lược indexing; thiết kế giao diện người dùng cho 6 loại trang chính.
- **Chương 4 – Triển khai và kiểm thử:** Trình bày chi tiết quy trình triển khai hệ thống với Docker Compose (5 service, health check, dependency chain); cấu hình môi trường và biến môi trường; quy trình CI/CD và giám sát (Prometheus metrics, AgentTrace observability, cảnh báo Slack/Email); lập lịch pipeline với Apache Airflow (4 DAGs); chiến lược kiểm thử đa cấp độ với 58 file test; kết quả kiểm thử chi tiết theo 10 nhóm chức năng; và đánh giá hiệu năng hệ thống.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các khái niệm nền tảng và công nghệ được sử dụng trong đề án. Phần đầu giới thiệu tổng quan về lĩnh vực nghiên cứu – nền tảng bất động sản trực tuyến và ứng dụng của chatbot trong lĩnh vực này. Phần tiếp theo đi sâu vào các khái niệm cốt lõi: Retrieval-Augmented Generation (RAG), hệ đa tác tử (Multi-Agent System), vector embedding, hybrid search và reranking. Phần cuối cùng giới thiệu chi tiết từng công nghệ được sử dụng trong hệ thống, từ frontend, backend, cơ sở dữ liệu đến AI/ML và data pipeline.

2.1. Tổng quan về lĩnh vực nghiên cứu

2.1.1. Nền tảng bất động sản trực tuyến

Các nền tảng bất động sản trực tuyến đóng vai trò trung gian kết nối người mua, người bán và nhà đầu tư. Tại Việt Nam, batdongsan.com.vn là cổng thông tin bất động sản lớn nhất với hàng chục nghìn tin đăng mới mỗi ngày, bao gồm cả tin bán và cho thuê với đa dạng loại hình: căn hộ chung cư, nhà riêng, nhà mặt phố, đất nền, biệt thự và nhà phố thương mại (shophouse). Các nền tảng này thường cung cấp chức năng đăng tin, tìm kiếm theo bộ lọc cơ bản (giá, diện tích, khu vực) và hiển thị thông tin chi tiết của từng bất động sản.

Tuy nhiên, các nền tảng hiện tại còn tồn tại nhiều hạn chế:

- **Tìm kiếm dựa trên từ khóa chính xác:** Hệ thống chỉ khớp chính xác từ khóa người dùng nhập với nội dung tin đăng, không hiểu được ngữ nghĩa và ngữ cảnh của truy vấn. Ví dụ, truy vấn “căn hộ thoáng mát, view đẹp” không thể được xử lý hiệu quả bằng SQL WHERE thông thường.
- **Thiếu tư vấn tổng hợp:** Người dùng phải tự tổng hợp thông tin từ nhiều tin đăng khác nhau để đưa ra quyết định. Không có cơ chế tự động phân tích xu hướng thị trường, so sánh giá giữa các khu vực, hay đánh giá tiềm năng đầu tư.
- **Không có hỗ trợ pháp lý:** Các vấn đề pháp lý (sổ đỏ, quy hoạch, thủ tục sang tên, thuế phí) là mối quan tâm hàng đầu của người mua nhưng không được các nền tảng đề cập đến một cách có hệ thống.
- **Dữ liệu không được tổ chức cho AI:** Dữ liệu trên các nền tảng được thiết kế để hiển thị cho con người đọc, không được cấu trúc và đánh chỉ mục để phục vụ các hệ thống truy xuất thông tin (information retrieval) và sinh văn bản (text generation).

2.1.2. Chatbot và trợ lý ảo trong bất động sản

Chatbot trong lĩnh vực bất động sản có tiềm năng tự động hóa quy trình tư vấn, giảm tải cho nhân viên môi giới và cung cấp dịch vụ 24/7 cho khách hàng. Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Models – LLMs) như GPT-4, Google Gemini, Claude đã mở ra khả năng xây dựng các chatbot thông minh, có khả năng hiểu ngôn ngữ tự nhiên và sinh phản hồi mạch lạc.

Tuy nhiên, LLM thuần túy có hai hạn chế lớn khi áp dụng vào lĩnh vực bất động sản:

1. **Hallucination (Ảo giác):** LLM có thể sinh ra thông tin không chính xác về giá cả, vị trí, hoặc tình trạng pháp lý của bất động sản – những thông tin đòi hỏi độ chính xác tuyệt đối trong giao dịch thực tế.
2. **Thông tin lỗi thời:** LLM được huấn luyện trên dữ liệu có thời điểm cắt (knowledge cutoff) nhất định, không thể cập nhật theo biến động hàng ngày của thị trường bất động sản.

Để khắc phục, các hệ thống chatbot hiện đại kết hợp LLM với kỹ thuật Retrieval-Augmented Generation (RAG) – cho phép mô hình truy xuất thông tin từ cơ sở dữ liệu ngoài trước khi sinh phản hồi. Cách tiếp cận này đảm bảo phản hồi dựa trên dữ liệu thực tế, có thể kiểm chứng và luôn được cập nhật.

2.2. Các khái niệm cơ bản

2.2.1. Retrieval-Augmented Generation (RAG)

RAG là kỹ thuật kết hợp giữa truy xuất thông tin (Information Retrieval) và sinh văn bản (Text Generation), được đề xuất bởi Lewis et al. (2020). Quy trình RAG gồm ba bước chính:

1. **Truy xuất (Retrieve):** Khi nhận được câu hỏi của người dùng, hệ thống chuyển câu hỏi thành vector embedding (sử dụng cùng mô hình embedding với dữ liệu) và tìm kiếm k đoạn văn bản (chunks) có độ tương đồng cao nhất trong cơ sở dữ liệu vector. Phép đo tương đồng thường dùng là cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|} \quad (2.1)$$

trong đó $\mathbf{q}$ là vector embedding của câu truy vấn, $\mathbf{d}_i$ là vector embedding của chunk thứ i .

2. **Tăng cường (Augment):** Các chunks được truy xuất sẽ được đưa vào prompt của LLM như ngữ cảnh bổ sung (context). Prompt thường có cấu trúc: system instruction + retrieved context + user query. Việc này giúp LLM có thông tin thực tế để dựa vào khi sinh câu trả lời.
3. **Sinh (Generate):** LLM sinh câu trả lời dựa trên câu hỏi gốc và ngữ cảnh được cung cấp, kèm theo trích dẫn nguồn (citations) cho phép người dùng kiểm chứng thông tin.

Ưu điểm của RAG:

- **Giảm thiểu ảo giác:** Phản hồi được neo (grounded) vào dữ liệu thực tế thay vì chỉ dựa vào kiến thức nội tại của LLM.
- **Thông tin cập nhật:** Cơ sở dữ liệu vector có thể được cập nhật liên tục mà không cần fine-tune lại LLM.
- **Kiểm chứng được:** Mỗi phản hồi đi kèm danh sách nguồn tham khảo, cho phép người dùng xác minh.
- **Domain-specific:** Có thể dễ dàng áp dụng cho lĩnh vực chuyên biệt (bất động sản, y tế, pháp luật) bằng cách xây dựng kho dữ liệu riêng.

Các biến thể RAG trong đồ án:

- **Simple RAG:** Truy xuất đơn giản – query embedding → vector search → LLM generation. Được sử dụng làm fallback trong chatbot pipeline.
- **Agent-based RAG:** Mỗi agent chuyên biệt thực hiện truy xuất riêng cho domain của mình (property, market, legal, investment), sau đó kết quả được tổng hợp.
- **Multi-step RAG (Agent Service):** Sử dụng retrieval planner để lập kế hoạch truy xuất qua nhiều bước, mỗi bước có thể truy xuất từ các nguồn khác nhau.

2.2.2. Multi-Agent System (Hệ đa tác tử)

Hệ đa tác tử (Multi-Agent System – MAS) là kiến trúc trong đó nhiều agent (tác tử) chuyên biệt cùng phối hợp để giải quyết một nhiệm vụ phức tạp. Mỗi agent đảm nhận một vai trò riêng, có system prompt riêng, và có thể truy cập các công cụ (tools) khác nhau. Kiến trúc này phù hợp với bài toán tư vấn bất động sản vì một truy vấn của người dùng thường liên quan đến nhiều khía cạnh: tìm kiếm, thị trường, pháp lý và đầu tư.

Trong đồ án, hệ thống multi-agent được triển khai theo hai lớp:

Lớp 1: Pipeline nội bộ (backend/app/services/chatbot/)

Đây là pipeline chính phục vụ người dùng cuối, hoạt động theo cơ chế:

1. **Router** – Phân loại ý định người dùng. Hỗ trợ hai phương thức:

- *Keyword-based routing*: Sử dụng biểu thức chính quy (regex) và từ khóa tiếng Việt đã chuẩn hóa (không dấu) để nhận diện ý định. Ví dụ: các từ khóa “pháp lý”, “so do”, “cong chung” kích hoạt `legal_advisor`; “dau tu”, “loi nhuan”, “sinh loi” kích hoạt `investment_advisor`. Phương thức này hoạt động không cần API key, đảm bảo tính sẵn sàng của hệ thống.
- *Gemini-based routing*: Khi `GEMINI_API_KEY` được cấu hình, sử dụng Google Gemini 2.0 Flash để phân loại ý định với độ chính xác cao hơn. Router gửi prompt chứa danh sách intent có thể và yêu cầu Gemini trả về JSON với intent và target agents.

2. **Orchestrator** – Nhận kết quả routing, chọn các agent tương ứng và gọi chúng song song qua `asyncio.gather()`. Sau khi tất cả agent hoàn thành, orchestrator tổng hợp kết quả: ghép nội dung, gộp danh sách nguồn, loại bỏ suggested actions trùng lặp.

3. **4 Agent chuyên biệt** – Mỗi agent nhận query, routing decision và database session, thực hiện truy xuất riêng và trả về `AgentResult`:

- **Property Search Agent (agents/property.py)**: Trích xuất bộ lọc từ truy vấn (giá, diện tích, khu vực, loại hình) bằng `extract_search_filters()`, thực hiện hybrid search trên bảng listings, định dạng kết quả thành danh sách bất động sản kèm giải thích.
- **Market Analysis Agent (agents/market.py)**: Truy vấn thống kê thị trường (giá trung bình, số lượng tin, phân bố) từ bảng listings, cung cấp phân tích xu hướng theo khu vực và loại hình.
- **Legal Advisor Agent (agents/legal.py)**: Tìm kiếm trong legal knowledge base (articles có category liên quan đến pháp lý) để cung cấp thông tin về thủ tục, quy định và lưu ý pháp lý.
- **Investment Advisor Agent (agents/investment.py)**: Phân tích khả năng sinh lời dựa trên giá, vị trí và xu hướng thị trường; đưa ra đánh giá sơ bộ về tiềm năng đầu tư.

Lớp 2: Agent Service (agent_service/)

Microservice riêng biệt chạy trên cổng 8100, sử dụng LangGraph StateGraph để điều phối multi-agent workflow với 8 node tuần tự:

1. **Context Builder** – Xây dựng ngữ cảnh hội thoại từ lịch sử chat và user preferences.
2. **Readiness Checker** – Kiểm tra trạng thái sẵn sàng của các nguồn dữ liệu (listings, projects, articles, legal_kb) qua bảng source_readiness.
3. **Router** – Phân loại ý định và chọn target agents.
4. **Retrieval Planner** – Lập kế hoạch truy xuất đa bước: xác định cần truy vấn những nguồn nào, với bộ lọc gì, độ ưu tiên ra sao.
5. **Specialist Agents** – Chạy song song các agent được chọn (property_search, market_analysis, legal_advisor, investment_advisor), mỗi agent sử dụng LLM (Gemini) kết hợp với tools (retrieval, market_stats).
6. **Synthesizer** – Tổng hợp kết quả từ các specialist agents thành phản hồi cuối cùng, đảm bảo tính mạch lạc và đầy đủ.
7. **Safety Validator** – Kiểm tra phản hồi về các tiêu chí: groundedness (có căn cứ), helpfulness (hữu ích), safety (an toàn), citation_quality (chất lượng trích dẫn).
8. **Memory Proposals** – Trích xuất sở thích người dùng từ hội thoại (ví dụ: “thích căn hộ Quận 7”, “ngân sách dưới 3 tỷ”) và tạo MemoryProposal để lưu vào hệ thống.

Cơ chế ghi nhớ người dùng (Memory System)

Hệ thống tích hợp cơ chế ghi nhớ thông qua hai bảng trong cơ sở dữ liệu:

- **UserPreference:** Lưu trữ sở thích đã được xác nhận của người dùng dưới dạng key-value JSON (ví dụ: preferred_city = "Ho Chi Minh", budget_max = 3.0). Các preference có độ tin cậy (confidence score) và nguồn gốc (source: user, agent_proposal, system).
- **MemoryProposal:** Lưu trữ các đề xuất sở thích được agent trích xuất từ hội thoại nhưng chưa được xác nhận. Mỗi proposal có trạng thái (pending, accepted, rejected) và có thể yêu cầu người dùng xác nhận.

Một số preference key được auto-apply khi confidence ≥ 0.8 (preferred_city, preferred_district, preferred_property_type, budget_max, budget_min). Các key khác cần người dùng xác nhận thông qua API POST /preferences/memory-proposals/{id}/accept.

2.2.3. Vector Embedding và Tìm kiếm ngữ nghĩa

Vector Embedding

Vector embedding là kỹ thuật chuyển đổi văn bản thành một vector số thực d -chiều trong không gian Euclidean, sao cho các văn bản có ngữ nghĩa tương đồng sẽ nằm gần nhau trong không gian này. Khoảng cách giữa hai vector thường được đo bằng cosine similarity hoặc Euclidean distance (L2).

Đồ án sử dụng mô hình **BGE-M3** (BAAI/bge-m3) – một mô hình embedding đa ngôn ngữ mã nguồn mở, hỗ trợ hơn 100 ngôn ngữ bao gồm tiếng Việt. Các đặc điểm kỹ thuật của BGE-M3 trong đồ án:

- **Không gian embedding:** 1024 chiều.
- **Normalize embeddings:** True (chuẩn hóa về unit vector để dùng cosine similarity hiệu quả).
- **Max sequence length:** 8192 tokens.
- **Batch size:** 16 văn bản mỗi lần encode.
- **Retry policy:** 3 lần thử với exponential backoff.
- **Device:** Tự động chọn CPU/GPU dựa trên cấu hình.

Mô hình được sử dụng cho hai mục đích: (1) tạo embedding cho toàn bộ dữ liệu (listings, projects, articles, legal docs) trong quá trình indexing, và (2) tạo embedding cho câu truy vấn của người dùng trong quá trình tìm kiếm. Cùng một mô hình được dùng cho cả indexing và querying để đảm bảo tính nhất quán của không gian vector.

Tìm kiếm ngữ nghĩa (Semantic Search)

Tìm kiếm ngữ nghĩa sử dụng vector embedding để tìm các đoạn văn bản có nội dung tương đồng với câu truy vấn, ngay cả khi không có từ khóa chung. Đồ án sử dụng **pgvector** – extension của PostgreSQL – để lưu trữ và tìm kiếm vector. Cụ thể:

- **Kiểu dữ liệu:** `VECTOR(1024)` – lưu vector 1024 chiều.
- **Độ đo:** Cosine distance (`<=>` operator trong pgvector).
- **Chỉ mục:** HNSW (Hierarchical Navigable Small World) – cấu trúc chỉ mục xấp xỉ cho tìm kiếm k-NN hiệu quả với tham số $m = 16$ (số kết nối mỗi node) và $ef_construction = 64$ (kích thước danh sách động khi xây dựng).

2.2.4. Hybrid Search và Reranking

Hybrid Search (Tìm kiếm lai)

Hybrid Search kết hợp hai phương pháp tìm kiếm bổ trợ:

1. **Structured Filtering (SQL WHERE):** Áp dụng các bộ lọc có cấu trúc trên các trường quan hệ – listing_type (sale/rent), property_type, city, district, price (min/max), area (min/max), bedrooms. Điều này đảm bảo kết quả chính xác về mặt thuộc tính.
2. **Semantic Search (pgvector kNN):** Sử dụng vector embedding để tìm các chunks có nội dung tương đồng ngữ nghĩa với câu truy vấn. Điều này cho phép tìm thấy các bất động sản phù hợp với mô tả mềm (“view đẹp”, “gần trường học”, “khu an ninh”).

Hai phương pháp được kết hợp trong cùng một truy vấn SQL: đầu tiên lọc theo điều kiện WHERE trên bảng listings, sau đó join với bảng chunks và sắp xếp theo cosine distance trên cột embedding. Kết quả cuối cùng là danh sách k bất động sản vừa thỏa mãn bộ lọc cứng, vừa có nội dung mô tả phù hợp ngữ nghĩa.

Reranking (Xếp hạng lại)

Sau khi có tập kết quả ứng viên từ hybrid search, kỹ thuật reranking được áp dụng để sắp xếp lại theo độ liên quan thực tế. Đồ án sử dụng **Cohere Rerank Multilingual v3.0** – mô hình Cross-Encoder chuyên dụng:

- **Nguyên lý:** Không giống như Bi-Encoder (mã hóa query và document riêng biệt rồi so sánh), Cross-Encoder xử lý cặp (query, document) đồng thời qua cùng một mạng, cho phép mô hình học được mối tương tác tinh tế giữa chúng. Điều này cho độ chính xác cao hơn đáng kể, đặc biệt với các truy vấn phức tạp.
- **Cách sử dụng trong đồ án:** Gọi Cohere API với query gốc và danh sách documents (nội dung các chunks đã truy xuất), nhận về relevance score cho từng cặp. Kết quả được sắp xếp lại theo score giảm dần và lấy top_n = 5 kết quả tốt nhất. Reranking được áp dụng tùy chọn (khi COHERE_API_KEY được cấu hình).

Redis Caching

Để giảm độ trễ cho các truy vấn lặp lại, hệ thống tích hợp Redis caching ở hai cấp độ:

- **Embedding cache:** Cache vector embedding của câu truy vấn (key là hash của query text + model name + dimension). Điều này đặc biệt hữu ích khi nhiều agent cùng truy xuất với cùng một query.
- **Search result cache:** Cache kết quả hybrid search (key là hash của query + filters). Thời gian sống (TTL) mặc định được cấu hình để cân bằng giữa freshness và hiệu năng.

2.3. Các công nghệ sử dụng

2.3.1. FastAPI

FastAPI là web framework Python hiện đại, được xây dựng trên nền tảng Starlette (ASGI) và Pydantic. Các đặc điểm chính:

- **Hiệu năng cao:** FastAPI là một trong những Python framework nhanh nhất, ngang hàng với Node.js và Go, nhờ kiến trúc bất đồng bộ (async/await) và ASGI.
- **Async database:** Tích hợp SQLAlchemy 2.0 async với asyncpg driver, cho phép xử lý database I/O không chặn (non-blocking). Connection pool được cấu hình với pool_size=20 và max_overflow=10.
- **Validation tự động:** Pydantic v2 cung cấp validation dữ liệu đầu vào/đầu ra với type hints, tự động sinh JSON Schema và tài liệu OpenAPI.
- **Dependency injection:** Cơ chế Depends() cho phép tái sử dụng logic (database session, authentication, rate limiting) một cách sạch sẽ.
- **Tài liệu tương tác:** Swagger UI (/docs) và ReDoc được sinh tự động từ code.

Trong đồ án, FastAPI được sử dụng cho cả Backend API (cổng 8000) và Agent Service (cổng 8100).

2.3.2. Next.js

Next.js là React framework hỗ trợ Server-Side Rendering (SSR), Static Site Generation (SSG), Incremental Static Regeneration (ISR) và App Router (từ phiên bản 13+). Đồ án sử dụng Next.js 16 với React 19:

- **App Router:** Tổ chức trang theo cấu trúc thư mục app/, mỗi thư mục là một route segment. Hỗ trợ layouts lồng nhau, loading states, error boundaries.

- **Server Components:** Mặc định các component là React Server Components (RSC), giảm lượng JavaScript gửi về client.
- **Tailwind CSS v4:** Utility-first CSS framework cho phép xây dựng giao diện nhanh, nhất quán, không cần viết CSS thủ công.
- **SSR cho SEO:** Các trang danh sách và chi tiết bất động sản được render phía server, đảm bảo nội dung có sẵn cho crawler của công cụ tìm kiếm.
- **API client:** File `lib/api.ts` định nghĩa typed API client với các hàm gọi backend FastAPI, sử dụng fetch API với JWT token trong Authorization header.

2.3.3. PostgreSQL + pgvector

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được sử dụng làm cơ sở dữ liệu chính của hệ thống. Extension **pgvector** bổ sung khả năng lưu trữ và tìm kiếm vector, biến PostgreSQL thành vector database:

- **Kiểu dữ liệu VECTOR:** Lưu vector d -chiều (trong đồ án: 1024) dưới dạng chuỗi bytes tối ưu.
- **Phép toán tương đồng:** Hỗ trợ 3 độ đo khoảng cách:
 - $\langle - \rangle$ – Euclidean distance (L2).
 - $\langle \# \rangle$ – Negative inner product.
 - $\langle = \rangle$ – Cosine distance (được sử dụng trong đồ án).
- **HNSW Index:** Chỉ mục xấp xỉ cho tìm kiếm k-NN nhanh. Trong đồ án, HNSW index được tạo trực tiếp trong model definition qua `Index()` với `postgresql_using="hnsw"` và `postgresql_ops={"embedding": "vector_cosine_ops"}`.
- **Ưu điểm so với dedicated vector DB:** Dữ liệu quan hệ (listings, users, chats) và dữ liệu vector (chunks, embeddings) nằm trong cùng một hệ thống, cho phép JOIN, filter, và transaction ACID trên cả hai loại dữ liệu. Giảm độ phức tạp vận hành (chỉ cần quản lý một database).

2.3.4. LangGraph

LangGraph là thư viện của LangChain cho phép xây dựng các ứng dụng multi-agent có trạng thái (stateful) dưới dạng đồ thị có hướng (Directed Graph). Các khái niệm chính:

- **StateGraph:** Đồ thị trạng thái trong đó mỗi node nhận state object, xử lý và trả về state đã cập nhật. State được định nghĩa bởi TypedDict hoặc Pydantic model.
- **Nodes:** Các hàm xử lý (có thể là sync hoặc async) đại diện cho một bước trong pipeline. Trong đồ án, Agent Service có 8 nodes.
- **Edges:** Kết nối có hướng giữa các nodes, xác định luồng xử lý. Có hai loại: normal edge (luôn đi từ A đến B) và conditional edge (chọn node tiếp theo dựa trên state).
- **Checkpointing:** Lưu trạng thái sau mỗi bước, cho phép resume từ điểm dừng và debug.

Trong đồ án, LangGraph được sử dụng trong Agent Service (`agent_service/graph/world`) với StateGraph được compile và chạy qua `chat_graph.invoke()`. State object (`AgentGraphS`) theo dõi toàn bộ quá trình xử lý: request gốc, intent, retrieval plan, kết quả từng agent, final response, sources, trace steps, warnings.

Pipeline nội bộ trong backend (`chatbot/orchestrator.py`) không sử dụng LangGraph mà dùng cơ chế đơn giản hơn: router chọn agents, `asyncio.gather()` chạy song song, và combine kết quả.

2.3.5. Google Gemini

Google Gemini 2.0 Flash là mô hình ngôn ngữ lớn đa năng của Google, hỗ trợ đa ngôn ngữ (bao gồm tiếng Việt) với khả năng sinh văn bản, phân loại và tạo embedding. Trong đồ án, Gemini được sử dụng cho ba nhiệm vụ:

1. **Intent Router (Gemini-based routing):** Khi `GEMINI_API_KEY` được cấu hình, router sử dụng Gemini để phân loại ý định người dùng thay vì keyword matching. Prompt yêu cầu Gemini phân tích query và trả về JSON với intent và danh sách target agents phù hợp. Điều này cho độ chính xác cao hơn với các truy vấn phức tạp hoặc mơ hồ.
2. **LLM cho Agent Service:** Trong Agent Service, Gemini được sử dụng làm LLM chính cho các specialist agents (qua GeminiClient wrapper trong `agent_service/llm/gemini`). Mỗi agent nhận system prompt mô tả vai trò, context từ retrieval, và sinh phản hồi chuyên biệt.
3. **LLM Judge:** Gemini được sử dụng để đánh giá chất lượng phản hồi của chatbot qua 5 tiêu chí: groundedness (tính có căn cứ), helpfulness (tính hữu ích), citation_quality (chất lượng trích dẫn), safety (an toàn), và trace_completeness (tính đầy đủ của trace). Prompt judge được viết bằng tiếng Việt và yêu cầu output JSON.

Cơ chế fallback: Khi GEMINI_API_KEY không được cấu hình, hệ thống tự động chuyển sang keyword-based routing cho intent classification. Các agent trong pipeline nội bộ có thể hoạt động không cần LLM (dùng template-based response từ dữ liệu truy xuất). Điều này đảm bảo hệ thống luôn hoạt động được ngay cả khi không có kết nối đến Gemini API.

2.3.6. BGE-M3 Embedding

BGE-M3 (BAAI/bge-m3) là mô hình embedding đa ngôn ngữ mã nguồn mở từ BAAI (Beijing Academy of Artificial Intelligence), hỗ trợ hơn 100 ngôn ngữ bao gồm tiếng Việt. Đây là mô hình embedding chính của đề án, được chọn vì các lý do:

- **Chất lượng cao cho tiếng Việt:** BGE-M3 được huấn luyện trên corpus đa ngôn ngữ lớn, cho chất lượng embedding tốt với văn bản tiếng Việt – vượt trội so với Gemini Embedding trong các thử nghiệm nội bộ.
- **Không phụ thuộc API:** Là mô hình mã nguồn mở, chạy local qua thư viện `sentence-transformers` không giới hạn về số lượng request hay chi phí API. Điều này đặc biệt quan trọng khi cần tạo embedding cho hàng trăm nghìn chunks.
- **Hỗ trợ batch processing:** BGEEmbedder trong đề án (`rag/embeddings.py`) hỗ trợ encode theo batch (`batch_size=16`), chạy async qua `asyncio.to_thread()` để không chặn event loop.
- **Multi-granularity:** BGE-M3 hỗ trợ cả dense embedding (1024 chiều), sparse embedding (lexical), và multi-vector – mặc dù đề án hiện chỉ sử dụng dense embedding.

2.3.7. Cohere Rerank

Cohere Rerank Multilingual v3.0 là mô hình Cross-Encoder chuyên dụng cho bài toán xếp hạng lại kết quả truy xuất. So với cách tiếp cận Bi-Encoder (tạo embedding riêng cho query và document):

- **Kiến trúc Cross-Encoder:** Xử lý cặp (query, document) đồng thời qua cùng một mạng transformer, cho phép mô hình học được mối tương tác tinh tế giữa chúng. Điều này cho độ chính xác cao hơn đáng kể so với so sánh cosine similarity của Bi-Encoder.
- **Đa ngôn ngữ:** Hỗ trợ tiếng Việt và nhiều ngôn ngữ khác, phù hợp với dữ liệu bất động sản tiếng Việt.

- **Sử dụng trong đồ án:** Gọi qua HTTP API (httpx client), gửi query và danh sách documents, nhận relevance score cho từng cặp. Kết quả được lọc theo RERANK_TOP_N (mặc định: 5).
- **Cấu hình linh hoạt:** Reranking có thể bật/tắt qua biến môi trường (RERANK_PROVIDER), cho phép hệ thống hoạt động không cần Cohere API key.

2.3.8. Redis

Redis (Remote Dictionary Server) là cơ sở dữ liệu key-value in-memory, được sử dụng làm cache trong đồ án:

- **Embedding cache:** Lưu vector embedding của query để tránh encode lại. Key format: `embedding:{provider}:{model}:{dimension}:{hash}`.
- **Search cache:** Lưu kết quả hybrid search để phục vụ truy vấn lặp lại. Sử dụng JSON serialization qua `JsonCache` class.
- **Session store:** Lưu trữ session state tạm thời cho chat.
- **Health check:** Docker healthcheck sử dụng `redis-cli ping`.

2.3.9. Apache Airflow

Apache Airflow là nền tảng quản lý workflow mã nguồn mở, cho phép định nghĩa, lập lịch và giám sát các pipeline dữ liệu dưới dạng DAG (Directed Acyclic Graph). Trong đồ án, Airflow quản lý 4 DAGs:

- **daily_listings_dag:** Chạy hàng ngày lúc 02:00 (giờ Hồ Chí Minh). Gồm 8 tasks: `crawl_sale_urls` → `crawl_sale_details` → `ingest_sale` (cùng lúc với `crawl_rent_urls` → `crawl_rent_details` → `ingest_rent`) → `mark_active` → `chunk_and_embed`. Sử dụng `watermark (-since)` để chỉ crawl tin mới.
- **weekly_projects_dag:** Chạy hàng tuần vào Chủ nhật 03:00. Crawl thông tin dự án bất động sản (tên, chủ đầu tư, vị trí, quy mô, tiện ích) và nạp vào bảng `projects`.
- **weekly_news_dag:** Chạy hàng tuần vào Chủ nhật 04:00. Crawl tin tức và bài viết phân tích thị trường, nạp vào bảng `articles`.
- **monthly_legal_kb_dag:** Chạy hàng tháng vào ngày 1 lúc 05:00. Xử lý lại toàn bộ tài liệu pháp lý từ thư mục `data/knowledge/`, bao gồm cả HTML và PDF, tạo chunks và embedding mới.

Mỗi DAG được cấu hình retry 3 lần với exponential backoff (5 phút → 10 phút → 20 phút, tối đa 30 phút). Cơ chế cảnh báo qua Slack và email được tích hợp khi task thất bại. Metrics được ghi vào bảng pipeline_runs để theo dõi hiệu suất.

2.3.10. Playwright

Playwright là thư viện tự động hóa trình duyệt do Microsoft phát triển, hỗ trợ Chromium, Firefox và WebKit. Trong đồ án, Playwright được sử dụng để crawl dữ liệu từ batdongsan.com.vn:

- **Stealth mode:** Cấu hình trình duyệt với các kỹ thuật chống phát hiện (stealth) để tránh bị chặn bởi cơ chế anti-bot của website.
- **Parallel workers:** Sử dụng đa luồng (4 workers) để crawl song song nhiều trang, tăng tốc độ thu thập dữ liệu.
- **Phân chia module:** Crawler được tổ chức thành các module riêng biệt cho từng loại nội dung: crawler/sale/ (tin bán), crawler/rent/ (tin cho thuê), crawler/project (dự án), crawler/news/ (tin tức). Mỗi module có hai bước: crawl URL và crawl chi tiết.
- **CSV output:** Dữ liệu được ghi vào file CSV với encoding UTF-8 BOM, hỗ trợ deduplication qua product_id và merge các file tạm.
- **Core infrastructure:** crawler/core/ chứa các thành phần dùng chung: csv_writer (ghi CSV với dedup), parser (parse thông tin cơ bản), listing_detail_parser (parse chi tiết listing với đầy đủ trường).

2.3.11. Các thư viện và công cụ khác

- **SQLAlchemy 2.0:** ORM cho Python với hỗ trợ async/await qua asyncpg driver. Được sử dụng để định nghĩa models, thực hiện truy vấn và quản lý schema.
- **Pydantic v2:** Thư viện validation dữ liệu sử dụng type hints. Được sử dụng cho request/response schemas trong FastAPI và internal contracts giữa các service.
- **bcrypt:** Thư viện hash mật khẩu với salt, được sử dụng trong authentication flow.
- **PyJWT:** Thư viện tạo và xác thực JSON Web Token, với thuật toán HS256 và thời hạn 24 giờ.
- **HTTPX:** HTTP client bất đồng bộ, được sử dụng để gọi Cohere Rerank API và internal Agent Service API.

- **Docker & Docker Compose:** Containerization cho toàn bộ hệ thống với 5 service, health checks, và volume mounts.
- **Prometheus:** Hệ thống giám sát và thu thập metrics, tích hợp qua endpoint `/api/v1/metrics` với các counter cho chat requests, retrieval latency và pipeline runs.

CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương này trình bày toàn bộ quá trình phân tích và thiết kế hệ thống Real Estate Chatbot Platform. Nội dung bao gồm: phân tích yêu cầu chức năng và phi chức năng, thiết kế kiến trúc tổng thể theo mô hình microservices, thiết kế chi tiết các module thành phần (frontend, backend API, agent service, data pipeline), thiết kế cơ sở dữ liệu quan hệ kết hợp vector, thiết kế giao diện người dùng, quy trình triển khai với Docker và kết quả kiểm thử hệ thống.

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng

Hệ thống phục vụ ba nhóm đối tượng chính: người dùng cuối (người mua, người thuê, nhà đầu tư), quản trị viên hệ thống, và bản thân hệ thống (các tác vụ tự động). Dưới đây là các nhóm chức năng được phân tích chi tiết.

Nhóm chức năng dành cho người dùng cuối

Đây là nhóm chức năng cốt lõi, quyết định trải nghiệm của người dùng khi tương tác với nền tảng:

1. Xác thực và quản lý tài khoản:

- Đăng ký tài khoản mới với email và mật khẩu. Mật khẩu được hash bằng bcrypt trước khi lưu trữ.
- Đăng nhập bằng email/mật khẩu, hệ thống trả về JWT token có thời hạn 24 giờ. Token được lưu ở client (localStorage) và gửi kèm trong Authorization header cho các request tiếp theo.
- Người dùng chưa đăng nhập (anonymous) vẫn có thể sử dụng chatbot với giới hạn 20 lượt/ngày. Người dùng đã xác thực được nâng lên 200 lượt/ngày.
- Hồ sơ người dùng bao gồm: họ tên, email, số điện thoại, ảnh đại diện, quyền (is_admin, is_superuser).

2. Duyệt và tìm kiếm bất động sản:

- **Danh sách bất động sản bán:** Hiển thị dưới dạng lưới thẻ (card grid) với phân trang (pagination). Mỗi thẻ hiển thị: tiêu đề, giá, diện tích, địa chỉ (quận, thành phố), số phòng ngủ, loại tin (VIP/thường).

- **Danh sách bất động sản cho thuê:** Tương tự như trang bán nhưng dành riêng cho tin cho thuê.
- **Bộ lọc nâng cao:** Người dùng có thể lọc theo 10+ tiêu chí: loại tin (bán/cho thuê), loại hình bất động sản (căn hộ, nhà riêng, đất nền, biệt thự,...), thành phố, quận/huyện, khoảng giá (từ – đến), diện tích (từ – đến), số phòng ngủ, số phòng tắm, hướng nhà.
- **Tìm kiếm theo từ khóa:** Hỗ trợ tìm kiếm full-text trên các trường tiêu đề, mô tả, địa chỉ, quận, thành phố.
- **Sắp xếp kết quả:** Theo giá tăng dần/giảm dần, diện tích tăng dần/giảm dần, mới nhất.

3. Xem chi tiết bất động sản:

- Trang chi tiết hiển thị đầy đủ thông tin của một bất động sản: tiêu đề, mô tả chi tiết, giá (dạng số + text), đơn giá theo m^2 , diện tích (dạng số + text), địa chỉ đầy đủ (phường/xã, quận/huyện, tỉnh/thành phố), tọa độ địa lý (latitude, longitude).
- Thông số kỹ thuật: số phòng ngủ, số phòng tắm, số tầng, hướng nhà, hướng ban công, mặt tiền, đường vào.
- Thông tin pháp lý và nội thất: tình trạng pháp lý (sổ đỏ, sổ hồng,...), tình trạng nội thất (đầy đủ, cơ bản, bàn giao thô).
- Thông tin liên hệ: tên và số điện thoại người đăng.
- **Bất động sản tương tự:** Hệ thống tự động gợi ý các bất động sản có đặc điểm tương đồng (cùng khu vực, cùng phân khúc giá, cùng loại hình).

4. Thống kê và phân tích thị trường:

- **Tổng quan thị trường:** Hiển thị các chỉ số chính: tổng số tin đăng đang hoạt động, giá trung bình, diện tích trung bình, số lượng tin bán và cho thuê, số thành phố và quận/huyện có dữ liệu.
- **Top khu vực:** Bảng xếp hạng các thành phố/quận có nhiều tin đăng nhất, có thể lọc theo loại tin.
- **Thống kê giá theo quận:** Với mỗi quận trong một thành phố, hiển thị số lượng tin, giá trung bình, giá thấp nhất, giá cao nhất và đơn giá trung bình/ m^2 .
- **Phân bố loại hình:** Thống kê số lượng tin theo từng loại hình bất động sản (căn hộ, nhà riêng, đất nền,...).

5. Tương tác với chatbot tư vấn:

- **Chat widget nổi:** Luôn hiển thị ở góc phải dưới màn hình, có thể mở rộng/thu gọn. Người dùng có thể đặt câu hỏi bất kỳ lúc nào.
- **Hội thoại đa lượt:** Hệ thống lưu trữ toàn bộ lịch sử hội thoại theo phiên (session). Người dùng có thể xem lại các phiên chat trước đó.
- **Phản hồi có trích dẫn nguồn:** Mỗi câu trả lời của chatbot đều kèm danh sách nguồn tham khảo (tên bất động sản, đường dẫn, điểm liên quan), cho phép người dùng kiểm chứng thông tin.
- **Gợi ý câu hỏi tiếp theo:** Sau mỗi phản hồi, chatbot đề xuất 2–3 câu hỏi liên quan để người dùng tiếp tục khai thác.
- **Đánh giá chất lượng:** Người dùng có thể đánh giá phản hồi (thích/không thích) và để lại nhận xét, giúp cải thiện chất lượng hệ thống.
- **Cá nhân hóa:** Hệ thống ghi nhớ sở thích của người dùng (khu vực ưa thích, ngân sách, loại hình) và sử dụng chúng để cá nhân hóa phản hồi trong các phiên sau.

Nhóm chức năng chatbot đa tác tử

Đây là nhóm chức năng cốt lõi tạo nên sự khác biệt của hệ thống so với các nền tảng bất động sản truyền thống:

1. Phân loại ý định người dùng (Intent Routing):

- Hệ thống tự động phân tích câu hỏi tiếng Việt để xác định người dùng đang cần: tìm kiếm bất động sản, phân tích thị trường, tư vấn pháp lý, hay tư vấn đầu tư.
- **Hai cơ chế routing:** (1) Keyword-based – sử dụng regex và từ khóa tiếng Việt không dấu để nhận diện ý định, hoạt động không cần API key; (2) Gemini-based – sử dụng Google Gemini 2.0 Flash để phân loại với độ chính xác cao hơn, tự động kích hoạt khi có GEMINI_API_KEY.
- Router có thể chọn nhiều agent cùng lúc nếu câu hỏi liên quan đến nhiều lĩnh vực (ví dụ: “căn hộ Quận 7 dưới 3 tỷ, thủ tục pháp lý thế nào” sẽ kích hoạt cả Property Search và Legal Advisor).

2. Tìm kiếm bất động sản thông minh (Property Search):

- Tự động trích xuất bộ lọc có cấu trúc từ câu hỏi tiếng Việt: loại giao dịch (mua/thuê), loại hình (căn hộ, nhà,...), thành phố, quận, khoảng giá, diện tích. Ví dụ: “tìm

căn hộ Quận 7 dưới 5 tỷ, 2 phòng ngủ” được parse thành {listing_type: sale, property_type: apartment, district: Quận 7, max_price: 5, bedrooms: 2}.

- Thực hiện hybrid search kết hợp SQL filtering và pgvector semantic search. Kết quả được xếp hạng lại qua Cohere Rerank (nếu có API key) và trả về kèm giải thích lý do từng bất động sản phù hợp.

3. Phân tích thị trường (Market Analysis):

- Cung cấp số liệu thống kê thị trường theo thời gian thực: giá trung bình, giá/m², số lượng tin đăng, xu hướng biến động theo khu vực và loại hình.
- So sánh giữa các khu vực: giá trung bình tại Quận 7 so với Quận 2, giá căn hộ so với nhà riêng trong cùng khu vực.

4. Tư vấn pháp lý (Legal Advisor):

- Cung cấp thông tin về các vấn đề pháp lý thường gặp trong giao dịch bất động sản: sổ đỏ/sổ hồng, thủ tục sang tên, công chứng hợp đồng, các loại thuế và phí, quy hoạch, điều kiện vay ngân hàng.
- Thông tin được truy xuất từ Legal Knowledge Base – kho tài liệu pháp lý được xử lý và lập chỉ mục định kỳ hàng tháng.

5. Tư vấn đầu tư (Investment Advisor):

- Phân tích tiềm năng đầu tư dựa trên các yếu tố: vị trí, giá, xu hướng thị trường, tiện ích xung quanh, quy hoạch hạ tầng.
- Đánh giá sơ bộ về thanh khoản, khả năng cho thuê lại, tỷ suất sinh lời kỳ vọng và biên an toàn tài chính.

6. Tổng hợp và phản hồi (Synthesis):

- Khi nhiều agent cùng được kích hoạt, orchestrator gọi chúng song song và tổng hợp kết quả thành một phản hồi duy nhất, mạch lạc, tránh trùng lặp.
- Mỗi phản hồi đều được kiểm tra an toàn (safety validation) trước khi gửi đến người dùng.

7. Ghi nhớ và cá nhân hóa (Memory System):

- Chatbot tự động trích xuất sở thích người dùng từ hội thoại (ví dụ: “Tôi thích căn hộ ở Quận 7, ngân sách khoảng 3 tỷ”) và lưu dưới dạng MemoryProposal.

- Các sở thích có độ tin cậy cao ($\geq 80\%$) được tự động áp dụng. Các sở thích khác cần người dùng xác nhận.
- Sở thích đã xác nhận được sử dụng để cá nhân hóa phản hồi trong các phiên chat sau.

Nhóm chức năng quản trị và tự động hóa

1. Thu thập dữ liệu tự động:

- **Crawl tin đăng hàng ngày:** Tự động thu thập tin bán và cho thuê mới từ batdongsan.com.vn vào 02:00 giờ Hồ Chí Minh. Cơ chế watermark đảm bảo chỉ crawl dữ liệu mới, tránh trùng lặp.
- **Crawl dự án hàng tuần:** Thu thập thông tin dự án bất động sản (chủ đầu tư, quy mô, vị trí, tiện ích) vào Chủ nhật hàng tuần.
- **Crawl tin tức hàng tuần:** Thu thập bài viết phân tích thị trường, tin tức bất động sản.
- **Cập nhật kiến thức pháp lý hàng tháng:** Xử lý lại toàn bộ tài liệu pháp lý, bao gồm cả HTML và PDF.

2. Xử lý dữ liệu (ETL Pipeline):

- **Làm sạch:** Chuẩn hóa giá (tỷ, triệu, triệu/tháng), diện tích (m^2), phân loại loại hình bất động sản, chuẩn hóa địa chỉ.
- **Làm giàu:** Geocoding – chuyển địa chỉ thành tọa độ (latitude, longitude) qua Nominatim hoặc Goong API. Intent tagging – tự động gán nhãn nhu cầu (“gần trường học”, “view đẹp”, “an ninh”,...) dựa trên nội dung mô tả.
- **Chunking:** Chia văn bản thành các đoạn ngữ nghĩa (chunks) với 4 loại: overview (tổng quan), description (mô tả chi tiết), location (vị trí), intent_tags (nhu cầu). Mỗi chunk được thiết kế để có thể đứng độc lập về mặt ngữ nghĩa.
- **Embedding:** Tạo vector embedding 1024 chiều cho từng chunk sử dụng mô hình BGE-M3 (BAAI/bge-m3) với batch size 16 và retry policy 3 lần.
- **Nạp dữ liệu:** Upsert vào PostgreSQL: bảng listings cho dữ liệu quan hệ, bảng chunks cho dữ liệu vector. Sử dụng cơ chế batch processing để tối ưu hiệu năng.

3. Quản trị và giám sát hệ thống:

- **Admin Dashboard:** Giao diện quản trị với các tab: Pipeline Readiness (trạng thái các nguồn dữ liệu), Agent Traces (lịch sử xử lý của agent với latency và status), Chat Feedback (phản hồi người dùng), Memory Proposals (các đề xuất đang chờ xác nhận).
- **Observability:** Hệ thống tracing toàn diện cho phép theo dõi toàn bộ luồng xử lý của từng request chatbot: intent, agents được chọn, thời gian xử lý từng bước, kết quả retrieval, LLM calls, errors.
- **Monitoring:** Prometheus metrics endpoint cung cấp các chỉ số: số lượng request chat, thời gian phản hồi, số lượng pipeline runs, tỷ lệ lỗi.
- **Alerting:** Cơ chế cảnh báo qua Slack và email khi pipeline thất bại hoặc hiệu năng bất thường.

3.1.2. Yêu cầu phi chức năng

Bảng 3.1 tổng hợp các yêu cầu phi chức năng của hệ thống.

Tiêu chí	Yêu cầu chi tiết
Hiệu năng	• API listings phản hồi < 500ms với bộ lọc cơ bản, < 1000ms với bộ lọc phức tạp • Chatbot pipeline hoàn thành < 5s cho truy vấn thông thường, < 15s cho truy vấn phức tạp có multi-agent • Trang web tải trong < 3s (First Contentful Paint) trên kết nối 4G
Khả năng mở rộng	• Kiến trúc microservices: mỗi service có thể scale độc lập • Database connection pool: 20 connections + 10 overflow • Redis caching giảm tải database cho truy vấn lặp lại • Airflow DAGs hỗ trợ parallel task execution

Bảo mật	• Xác thực JWT với HS256, token hết hạn sau 24h • Mật khẩu hash bằng bcrypt với salt • Chống SQL injection qua SQLAlchemy ORM (parameterized queries) • Chống XSS qua Next.js auto-escaping • CORS whitelist qua biến môi trường CORS_ORIGINS • Internal API giữa backend và agent service bảo vệ bằng internal key
Độ tin cậy	• Fallback mechanism 3 cấp: multi-agent pipeline → simple RAG → thông báo an toàn • Retry policy cho embedding: 3 lần với exponential backoff • Retry policy cho Airflow tasks: 3 lần, 5–30 phút • Health check endpoints cho tất cả services • Database health check trước khi khởi động service

Khả năng bảo trì	• Mã nguồn phân chia module rõ ràng theo domain • Docstring tiếng Anh cho tất cả hàm/class • Type hints đầy đủ (Python) và TypeScript (frontend) • Hơn 55 file test bao phủ toàn bộ module • ESLint cho frontend, compileall cho Python
Tương thích ngôn ngữ	• Giao diện người dùng: 100% tiếng Việt • Chatbot: hiểu và phản hồi bằng tiếng Việt • Embedding: BGE-M3 tối ưu cho hơn 100 ngôn ngữ bao gồm tiếng Việt • URL slugs: tiếng Việt không dấu

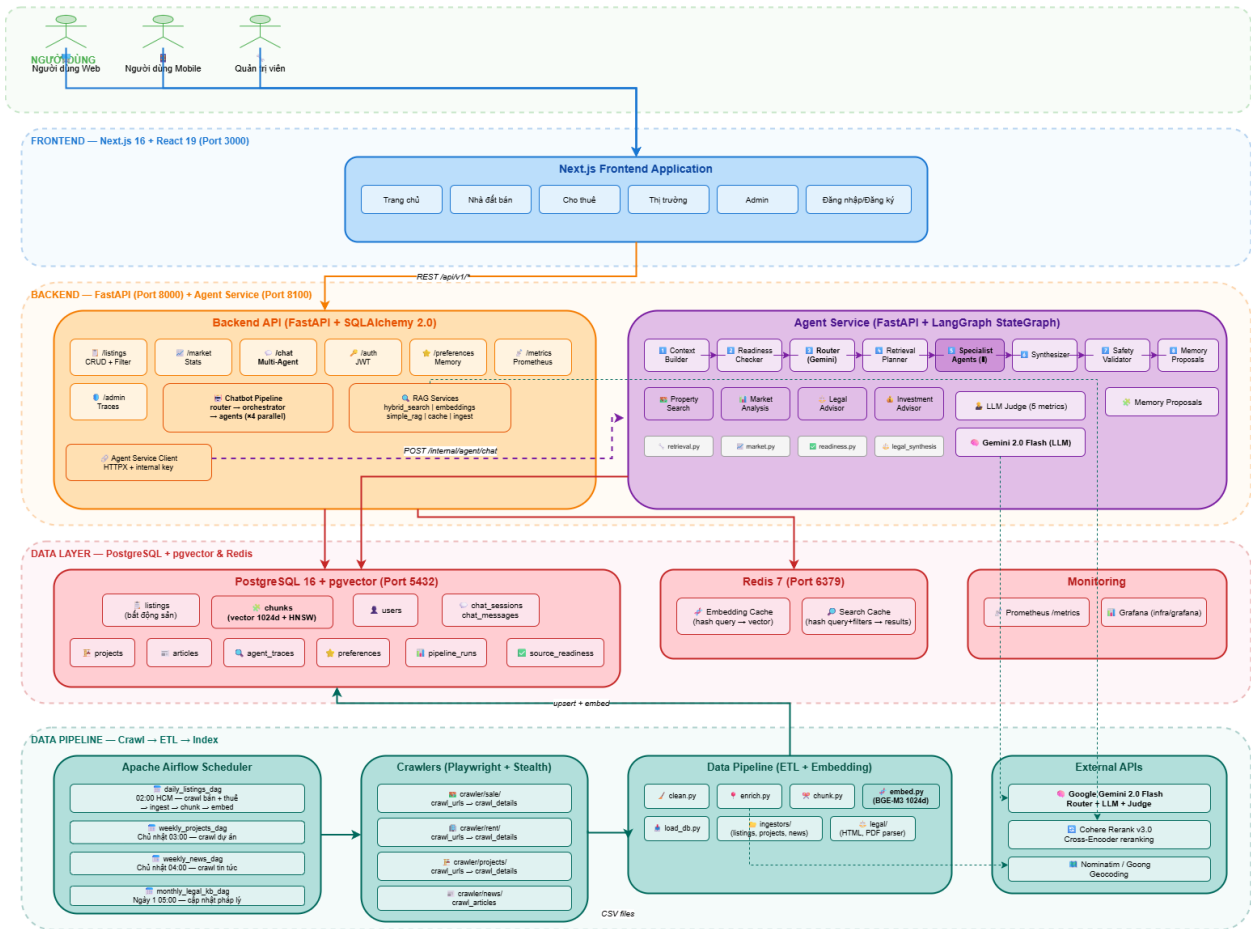
Bảng 3.1: Các yêu cầu phi chức năng của hệ thống

3.2. Thiết kế kiến trúc hệ thống

3.2.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc microservices, trong đó mỗi service đảm nhận một trách nhiệm riêng biệt và giao tiếp với nhau qua REST API. Hình 3.1 thể hiện kiến trúc tổng thể của toàn bộ hệ thống với 5 tầng chính.

KIẾN TRÚC HỆ THỐNG REAL ESTATE CHATBOT PLATFORM



Hình 3.1: Kiến trúc tổng thể hệ thống Real Estate Chatbot Platform

Tầng Người dùng (User Layer)

Người dùng tương tác với hệ thống qua trình duyệt web (desktop và mobile). Có hai nhóm người dùng chính:

- **Người dùng cuối:** Truy cập các trang duyệt bất động sản, xem thống kê thị trường và tương tác với chatbot tư vấn.
- **Quản trị viên:** Truy cập admin dashboard để giám sát pipeline, theo dõi agent traces và quản lý phản hồi người dùng.

Tầng Frontend – Next.js (Port 3000)

Frontend được xây dựng với Next.js 16 và React 19, sử dụng App Router để tổ chức các trang. Các đặc điểm thiết kế chính:

- **Server-Side Rendering (SSR):** Các trang danh sách và chi tiết bất động sản được render phía server, đảm bảo SEO và thời gian tải nhanh.

- **React Server Components (RSC):** Mặc định sử dụng Server Components để giảm JavaScript gửi về client.
- **Tailwind CSS v4:** Utility-first CSS framework đảm bảo giao diện nhất quán, responsive trên mọi thiết bị.
- **Typed API Client:** File `lib/api.ts` cung cấp các hàm gọi backend với TypeScript type safety, tự động gắn JWT token vào header.
- **Chat Widget:** Thành phần nổi (floating) luôn hiển thị, cho phép người dùng tương tác với chatbot ở bất kỳ trang nào.

Frontend chỉ giao tiếp trực tiếp với Backend API qua REST. Frontend không được phép gọi trực tiếp Agent Service – đây là nguyên tắc thiết kế quan trọng để đảm bảo bảo mật và khả năng bảo trì.

Tầng Backend API – FastAPI (Port 8000)

Backend API là cổng giao tiếp chính của toàn bộ hệ thống, đảm nhận mọi request từ frontend. Backend được tổ chức thành 4 lớp:

1. **Lớp Models (ORM):** Định nghĩa 14+ bảng dữ liệu với SQLAlchemy 2.0 async. Mỗi model ánh xạ trực tiếp đến một bảng trong PostgreSQL. Các mối quan hệ (relationships) được định nghĩa rõ ràng: Listing → Chunk (polymorphic), User → ChatSession → ChatMessage, User → UserPreference.
2. **Lớp Schemas (Pydantic):** Định nghĩa contract cho request/response. Pydantic v2 tự động validate dữ liệu đầu vào, sinh JSON Schema và tài liệu OpenAPI. Các schema được thiết kế tách biệt cho từng nghiệp vụ: listing (ListingCard, ListingDetail, PaginatedResponse), chat (ChatMessageRequest, ChatMessageResponse), auth (UserRegisterRequest, TokenResponse), admin (TraceSummary, PipelineReadinessItem).
3. **Lớp Routers (API Endpoints):** Xử lý HTTP request, gọi services tương ứng và trả về response. Backend có 7 router modules, mỗi router phụ trách một nhóm chức năng: listings, market, chat, auth, preferences, admin, metrics. Các router sử dụng dependency injection (Depends) để nhận database session và xác thực người dùng.
4. **Lớp Services (Business Logic):** Chứa toàn bộ logic nghiệp vụ, được chia thành 3 nhóm chính:

- **Chatbot Pipeline (`services/chatbot/`):** Pipeline multi-agent nội bộ với router (phân loại ý định), orchestrator (điều phối và tổng hợp), 4 agent chuyên biệt, context builder (xây dựng ngữ cảnh hội thoại), memory manager (quản lý ghi nhớ người dùng).
- **RAG Services (`services/rag/`):** Các dịch vụ truy xuất thông tin: hybrid search (kết hợp SQL + vector + rerank + cache), embeddings (BGE-M3 wrapper với batch processing và retry), cache (Redis caching cho embedding và kết quả tìm kiếm).
- **Agent Service Client (`services/agent_service/`):** HTTP client giao tiếp với Agent Service qua internal API, sử dụng internal key authentication.

Tầng Agent Service – FastAPI + LangGraph (Port 8100)

Agent Service là microservice riêng biệt, được thiết kế để xử lý các tác vụ AI/ML phức tạp mà không ảnh hưởng đến hiệu năng của Backend API chính. Service này sử dụng LangGraph để điều phối multi-agent workflow.

LangGraph StateGraph Workflow:

Agent Service triển khai một quy trình 8 bước dưới dạng StateGraph:

1. **Context Builder:** Xây dựng ngữ cảnh hội thoại từ lịch sử chat (ChatMessage) và sở thích người dùng (UserPreference). Ngữ cảnh này được đưa vào prompt của tất cả các bước sau.
2. **Readiness Checker:** Kiểm tra trạng thái sẵn sàng của các nguồn dữ liệu (listings, projects, articles, legal_kb) qua bảng source_readiness. Nếu một nguồn chưa sẵn sàng, hệ thống sẽ cảnh báo và điều chỉnh chiến lược truy xuất.
3. **Router:** Phân loại ý định người dùng và chọn danh sách agent cần kích hoạt. Router sử dụng Gemini 2.0 Flash với prompt được thiết kế riêng cho domain bất động sản tiếng Việt.
4. **Retrieval Planner:** Lập kế hoạch truy xuất đa bước – xác định cần truy vấn những nguồn dữ liệu nào (listings, projects, articles), với bộ lọc gì, độ ưu tiên ra sao. Kế hoạch được cấu trúc dưới dạng danh sách các RetrievalTask.
5. **Specialist Agents (chạy song song):** Các agent chuyên biệt được thực thi đồng thời:
 - **Property Search Agent:** Thực hiện hybrid search trên listings, trích xuất bộ lọc từ query, định dạng kết quả.

- **Market Analysis Agent:** Truy vấn thống kê thị trường, phân tích xu hướng.
- **Legal Advisor Agent:** Tìm kiếm trong legal knowledge base.
- **Investment Advisor Agent:** Phân tích tiềm năng đầu tư.

Mỗi agent nhận system prompt mô tả vai trò, context từ retrieval planner, và sử dụng Gemini để sinh phản hồi chuyên biệt.

6. **Synthesizer:** Tổng hợp kết quả từ các specialist agents thành một phản hồi cuối cùng mạch lạc. Synthesizer được hướng dẫn ưu tiên thông tin có độ tin cậy cao, loại bỏ trùng lặp và đảm bảo tính đầy đủ.
7. **Safety Validator:** Kiểm tra phản hồi cuối cùng về 5 tiêu chí: groundedness (có căn cứ từ nguồn), helpfulness (hữu ích cho người dùng), citation_quality (chất lượng trích dẫn), safety (an toàn, không đưa ra khẳng định pháp lý/tài chính tuyệt đối), trace_completeness (đầy đủ thông tin tracing).
8. **Memory Proposals:** Trích xuất sở thích tiềm năng của người dùng từ nội dung hội thoại và tạo MemoryProposal. Ví dụ: nếu người dùng nhiều lần hỏi về căn hộ Quận 7, hệ thống sẽ đề xuất lưu `preferred_district = "Quận 7"`.

Cơ chế đánh giá chất lượng (LLM Judge):

Sau mỗi phản hồi, hệ thống có thể kích hoạt LLM Judge (Gemini) để đánh giá chất lượng. Judge chấm điểm theo 5 metrics (thang 0.0–1.0) và cung cấp rationale cho từng điểm. Kết quả đánh giá được lưu vào `agent_traces` để phân tích và cải thiện hệ thống.

Tương tác giữa Backend và Agent Service:

- Backend gọi Agent Service qua HTTP client với internal key authentication.
- Agent Service nhận AgentChatRequest (query, session_id, user_id, conversation_history) và trả về AgentChatResponse (final_response, agents_used, sources, suggested_actions, trace_summary, full_trace).
- Backend lưu kết quả vào ChatMessage, AgentTrace và các bảng liên quan trước khi trả về frontend.
- Khi Agent Service không được kích hoạt (`CHATBOT_AGENT_SERVICE_ENABLED=false`) hoặc không khả dụng, Backend tự động fallback sang pipeline nội bộ.

Tầng Dữ liệu – PostgreSQL + pgvector & Redis

PostgreSQL 16 + pgvector (Port 5432):

PostgreSQL được chọn làm cơ sở dữ liệu chính nhờ khả năng lưu trữ đồng thời dữ liệu quan hệ và dữ liệu vector thông qua extension pgvector. Điều này giúp giảm độ phức tạp vận hành so với việc sử dụng một relational database và một vector database riêng biệt.

- **Dữ liệu quan hệ:** 14+ bảng bao gồm listings (bất động sản), users (người dùng), chat_sessions và chat_messages (hội thoại), projects (dự án), articles (bài viết), cùng các bảng observability (agent_traces, agent_trace_steps, agent_llm_calls, agent_retrieval_events, pipeline_runs, source_readiness, user_preferences, memory_proposals, chat_feedback).
- **Dữ liệu vector:** Bảng chunks lưu vector embedding 1024 chiều, phục vụ tìm kiếm ngữ nghĩa. HNSW index với tham số $m = 16$ và $ef_construction = 64$ đảm bảo tốc độ tìm kiếm k-NN nhanh trên tập dữ liệu lớn.
- **Thiết kế polymorphic cho chunks:** Cột parent_type và parent_id cho phép một bảng chunks duy nhất phục vụ nhiều loại dữ liệu nguồn (listings, projects, articles), thay vì phải tạo bảng chunks riêng cho từng loại.

Redis 7 (Port 6379):

Redis được sử dụng làm cache ở hai cấp độ:

- **Embedding Cache:** Lưu vector embedding của câu truy vấn, tránh phải encode lại. Key có dạng `embedding:{provider}:{model}:{dim}:{hash}`, cho phép cache hoạt động chính xác ngay cả khi thay đổi model.
- **Search Result Cache:** Lưu kết quả hybrid search cho các truy vấn lặp lại, giảm tải đáng kể cho PostgreSQL.

Tầng Data Pipeline & Tự động hóa

Đây là tầng vận hành ngầm, đảm bảo dữ liệu luôn được cập nhật và sẵn sàng cho các tầng phía trên.

Crawlers (Playwright + Stealth):

Hệ thống crawler được tổ chức thành 5 module, mỗi module phụ trách một loại nội dung:

- `crawler/core/`: Các thành phần dùng chung – CSV writer (với dedup), base parser, listing detail parser.

- `crawler/sale/`: Crawl tin bán (2 bước: crawl URLs → crawl details).
- `crawler/rent/`: Crawl tin cho thuê (tương tự).
- `crawler/projects/`: Crawl thông tin dự án bất động sản.
- `crawler/news/`: Crawl tin tức và bài viết phân tích.

Crawler sử dụng Playwright với chế độ stealth để tránh bị phát hiện, hỗ trợ 4 workers chạy song song để tăng tốc độ. Dữ liệu được ghi ra file CSV UTF-8 BOM và được deduplicate dựa trên `product_id`.

Data Pipeline (ETL + Embedding):

Pipeline xử lý dữ liệu gồm 5 bước chính, được thiết kế để chạy độc lập hoặc theo chuỗi:

1. **Clean (`clean.py`):** Parse và chuẩn hóa dữ liệu thô từ CSV. Các công việc chính: tách giá trị số từ text giá (“4,68 tỷ” → 4.68, unit: billion), tách diện tích (“75 m²” → 75), phân loại loại hình bất động sản, chuẩn hóa địa chỉ.
2. **Enrich (`enrich.py`):** Làm giàu dữ liệu với hai tác vụ: (1) Geocoding – gọi Nominatim/Goong API để chuyển địa chỉ text thành tọa độ (latitude, longitude); (2) Intent Tagging – phân tích nội dung mô tả để gán nhãn nhu cầu (“gần trường học”, “view đẹp”, “an ninh”, “pháp lý rõ”,...).
3. **Chunk (`chunk.py`):** Chia văn bản thành các đoạn ngữ nghĩa. Mỗi listing được chunk thành 4 loại: overview (tổng quan – tiêu đề, giá, diện tích, khu vực, pháp lý, nội thất), description (mô tả chi tiết), location (địa chỉ, quận, thành phố), intent_tags (nhu cầu phù hợp). Thiết kế này đảm bảo mỗi chunk có thể đứng độc lập và được truy xuất chính xác theo ngữ cảnh.
4. **Embed (`embed.py`):** Tạo vector embedding 1024 chiều cho từng chunk bằng mô hình BGE-M3. Sử dụng BGEmbedder class với batch size 16, retry 3 lần với exponential backoff, chạy async qua `asyncio.to_thread()` để không chặn event loop.
5. **Load (`load_db.py` + `ingestors/`):** Upsert dữ liệu đã xử lý vào PostgreSQL. Các ingestor module chuyên biệt cho từng loại dữ liệu (listings, projects, news, legal_kb) đảm bảo logic nạp phù hợp với schema của từng bảng.

Apache Airflow Scheduler:

Toàn bộ pipeline được lập lịch và giám sát bởi Apache Airflow với 4 DAGs:

- **daily_listings_dag (02:00 HCM):** Quy trình đầy đủ cho tin đăng: crawl_sale_urls → crawl_sale_details → ingest_sale (chạy song song với crawl_rent_urls → crawl_rent_details → ingest_rent) → mark_active → chunk_and_embed. Sử dụng watermark để chỉ xử lý dữ liệu mới.
- **weekly_projects_dag (Chủ nhật 03:00):** Crawl và nạp dự án bất động sản.
- **weekly_news_dag (Chủ nhật 04:00):** Crawl và nạp tin tức thị trường.
- **monthly_legal_kb_dag (Ngày 1, 05:00):** Xử lý lại toàn bộ tài liệu pháp lý từ data/knowledge

Mỗi DAG được cấu hình retry 3 lần với exponential backoff (5–30 phút), cảnh báo qua Slack và email khi thất bại, và ghi metrics vào bảng pipeline_runs.

3.3. Thiết kế cơ sở dữ liệu

3.3.1. Mô hình quan hệ

Hệ thống sử dụng PostgreSQL 16 với extension pgvector. Mô hình dữ liệu được thiết kế xoay quanh 3 nhóm bảng chính:

Nhóm bảng nghiệp vụ (Business Tables)

Đây là nhóm bảng lưu trữ dữ liệu cốt lõi của hệ thống:

- **listings – Bảng bất động sản:** Bảng quan trọng nhất với hơn 30 trường dữ liệu, lưu toàn bộ thông tin của một tin đăng. Khóa chính là id (INTEGER, auto-increment). Mỗi listing có product_id duy nhất từ nguồn crawl để deduplicate. Các chỉ mục (indexes) được tạo trên các trường thường xuyên dùng trong bộ lọc: listing_type, city, district, property_type, price, area, bedrooms, post_date, is_active.
- **projects – Bảng dự án:** Lưu thông tin dự án bất động sản: tên, chủ đầu tư, vị trí, quy mô (tổng số căn), khoảng giá, khoảng diện tích, trạng thái (sắp mở bán, đang bán, đã bàn giao), loại hình, tiện ích (mảng ARRAY), mô tả.
- **articles – Bảng bài viết:** Lưu tin tức và kiến thức pháp lý: tiêu đề, nội dung, danh mục, nguồn, ngày đăng, URL gốc, metadata.
- **users – Bảng người dùng:** Lưu thông tin tài khoản: email (unique), mật khẩu đã hash, họ tên, số điện thoại, avatar, trạng thái hoạt động, quyền (admin/superuser).

Nhóm bảng hội thoại (Chat Tables)

- **chat_sessions – Phiên hội thoại:** Mỗi phiên có id UUID, liên kết đến user (có thể NULL cho anonymous), tiêu đề tự động sinh từ tin nhắn đầu tiên.
- **chat_messages – Tin nhắn:** Mỗi tin nhắn thuộc về một session, có role (user/assistant/system), nội dung, agent đã xử lý, và metadata JSONB (chứa sources, citations, search context).

Nhóm bảng vector và tìm kiếm (Vector & Retrieval Tables)

- **chunks – Đoạn văn bản đã embedding:** Thiết kế polymorphic với parent_type (listing/project/answer) và parent_id. Mỗi chunk có chunk_type xác định loại nội dung (overview/description/location/introduction). Nội dung văn bản, embedding là VECTOR(1024). HNSW index trên cột embedding với cosine distance operator đảm bảo tìm kiếm k-NN nhanh.

Nhóm bảng observability & vận hành (Observability Tables)

- **agent_traces:** Lưu vết xử lý của mỗi request chatbot: intent, danh sách agents đã dùng, trace_summary, full_trace (JSONB), latency, status, graph_version, prompt_version, model_name.
- **agent_trace_steps:** Chi tiết từng bước trong trace: step_name, status, latency, input/output JSON.
- **agent_llm_calls:** Log mỗi lần gọi LLM: node_name, model_name, latency, token estimates.
- **agent_retrieval_events:** Log mỗi sự kiện truy xuất.
- **pipeline_runs:** Log mỗi lần chạy Airflow DAG: dag_id, run_id, status, thời gian, metrics.
- **source_readiness:** Trạng thái sẵn sàng của từng nguồn dữ liệu.
- **user_preferences:** Sở thích người dùng dạng key-value JSONB.
- **memory_proposals:** Đề xuất sở thích từ agent.
- **chat_feedback:** Phản hồi của người dùng về chất lượng chat.

3.3.2. Chiến lược indexing

- **Index cho listings:** Các B-tree index trên `listing_type`, `city`, `district`, `property_type`, `price`, `area`, `bedrooms`, `post_date`, `is_active` giúp tối ưu các truy vấn lọc và sắp xếp phổ biến. Composite index ngầm được PostgreSQL sử dụng khi nhiều điều kiện AND.
- **Index cho chunks:** HNSW index trên cột `embedding` với `vector_cosine_ops`. Đây là loại index xấp xỉ (approximate nearest neighbor) cho phép tìm kiếm k-NN trong không gian 1024 chiều với độ phức tạp $O(\log N)$ thay vì $O(N)$ của full scan. Tham số $m = 16$ (số kết nối mỗi node trong đồ thị HNSW) và $ef_construction = 64$ (kích thước dynamic candidate list) được chọn sau thử nghiệm để cân bằng giữa tốc độ build index và chất lượng tìm kiếm.
- **Index cho polymorphic queries:** Composite index trên (`parent_type`, `parent_id`) của bảng `chunks` giúp truy vấn nhanh tất cả chunks thuộc về một `listing/project/article`.

3.4. Thiết kế giao diện người dùng

Giao diện người dùng được thiết kế theo nguyên tắc “mobile-first” với Tailwind CSS v4, đảm bảo trải nghiệm nhất quán trên mọi kích thước màn hình. Hệ thống bao gồm 6 loại trang chính:

3.4.1. Trang chủ (Home Page)

Trang chủ đóng vai trò là điểm khởi đầu cho hành trình của người dùng:

- **Hero section:** Thanh tìm kiếm lớn, nổi bật ở trung tâm, cho phép người dùng nhập từ khóa và chọn loại tin (bán/cho thuê) ngay lập tức.
- **Danh mục phổ biến:** Các liên kết nhanh đến các loại hình bất động sản phổ biến (căn hộ, nhà riêng, đất nền,...).
- **Listing nổi bật:** Một số bất động sản tiêu biểu được hiển thị ngay trên trang chủ.
- **Chat widget:** Luôn hiển thị ở góc phải dưới, sẵn sàng cho người dùng đặt câu hỏi.

3.4.2. Trang danh sách bất động sản (Listing Pages)

Trang danh sách được chia thành hai khu vực chính:

- **Filter Panel (bên trái, 25% chiều rộng):** Chứa tất cả các bộ lọc: loại hình bất động sản (select), khoảng giá (range), diện tích (range), số phòng ngủ, số phòng tắm, hướng nhà, thành phố, quận/huyện. Các bộ lọc được áp dụng ngay khi người dùng thay đổi giá trị.

- **Listing Grid (bên phải, 75% chiều rộng):** Hiển thị danh sách bất động sản dưới dạng lưới thẻ (card). Mỗi thẻ hiển thị: tiêu đề, giá (định dạng tỷ/triệu), diện tích, địa chỉ, số phòng ngủ, badge (VIP/thường). Phía trên lưới có thanh sắp xếp (giá, diện tích, mới nhất) và tổng số kết quả. Phía dưới có phân trang.

3.4.3. Trang chi tiết bất động sản (Detail Page)

Trang chi tiết cung cấp cái nhìn toàn diện về một bất động sản:

- **Thông tin cơ bản:** Tiêu đề, giá, đơn giá/m², diện tích, địa chỉ đầy đủ.
- **Thông số kỹ thuật:** Bảng thông số với các trường: số phòng ngủ, phòng tắm, số tầng, hướng nhà, hướng ban công, mặt tiền, đường vào, pháp lý, nội thất.
- **Mô tả chi tiết:** Nội dung mô tả đầy đủ từ người đăng.
- **Thông tin liên hệ:** Tên và số điện thoại người đăng.
- **Bất động sản tương tự:** Section cuối trang hiển thị các bất động sản có đặc điểm tương đồng.

3.4.4. Trang thống kê thị trường (Market Dashboard)

Dashboard cung cấp bức tranh tổng quan về thị trường bất động sản:

- **Stats Cards:** 4–6 thẻ số liệu nổi bật: tổng số tin, giá trung bình, diện tích trung bình, số tin bán, số tin cho thuê, số thành phố.
- **Bảng xếp hạng:** Top thành phố và quận có nhiều tin đăng nhất.
- **Phân tích giá theo quận:** Bảng chi tiết với các cột: quận, số lượng tin, giá trung bình, giá thấp nhất, giá cao nhất, đơn giá/m².
- **Phân bố loại hình:** Thống kê số lượng theo từng loại hình bất động sản.

3.4.5. Trang quản trị (Admin Dashboard)

Admin dashboard phục vụ quản trị viên theo dõi và vận hành hệ thống:

- **Pipeline Readiness:** Trạng thái của từng nguồn dữ liệu (listings, projects, articles, legal_kb) – số lượng bản ghi, số chunks, lần cập nhật cuối, cảnh báo.
- **Agent Traces:** Danh sách các request chatbot gần đây với intent, agents đã dùng, latency, status. Có thể click vào từng trace để xem chi tiết từng bước.

- **Chat Feedback:** Phản hồi của người dùng về chất lượng chatbot, có thể lọc theo rating và issue_type.
- **Memory Proposals:** Các đề xuất sở thích đang chờ xác nhận, có thể accept/reject.

3.4.6. Chat Widget

Chat widget là thành phần giao diện đặc biệt, luôn hiển thị trên mọi trang:

- **Trạng thái thu gọn:** Nút tròn có biểu tượng chat, nổi ở góc phải dưới màn hình.
- **Trạng thái mở rộng:** Panel chat chiếm khoảng 400px chiều rộng, 600px chiều cao, gồm:
 - Header: Tên chatbot, nút thu gọn/đóng.
 - Body: Lịch sử tin nhắn dạng bubble (user bên phải, assistant bên trái), tự động scroll xuống tin mới nhất. Mỗi phản hồi của assistant hiển thị kèm sources (có thể click để mở link gốc) và suggested actions.
 - Footer: Input text + nút gửi. Hỗ trợ gửi bằng Enter.
- **Loading state:** Hiệu ứng typing indicator khi chatbot đang xử lý.
- **Feedback:** Sau mỗi phản hồi, hiển thị nút like/dislike để người dùng đánh giá.

CHƯƠNG 4. TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày chi tiết quá trình triển khai hệ thống lên môi trường vận hành và chiến lược kiểm thử toàn diện được áp dụng trong đề án. Phần đầu mô tả kiến trúc triển khai với Docker Compose, cấu hình từng service, quy trình build và chạy hệ thống. Phần tiếp theo trình bày chiến lược kiểm thử đa tầng: từ kiểm thử đơn vị (unit test), kiểm thử tích hợp (integration test) đến kiểm thử hệ thống (system test) và kiểm thử chấp nhận (acceptance test). Cuối cùng, chương tổng hợp kết quả kiểm thử, các vấn đề đã phát hiện và hướng khắc phục.

4.1. Triển khai hệ thống với Docker

4.1.1. Kiến trúc triển khai

Hệ thống được triển khai theo mô hình container hóa với Docker Compose, bao gồm 5 service chính hoạt động trong cùng một mạng nội bộ (internal network). Mỗi service được đóng gói trong một Docker image riêng, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường vận hành. Hình 4.1 mô tả tổng quan kiến trúc triển khai.

Docker Compose – Real Estate Chatbot Platform				
Frontend	Backend	Agent Service	PostgreSQL	Redis
Next.js 16 Port 3000	FastAPI Port 8000	FastAPI + LangGraph Port 8100	16 + pgvector Port 5432	7 Alpine Port 6379
Internal Network: realestate_network				
Volumes: pgdata, redisdata				

Hình 4.1: Kiến trúc triển khai Docker Compose

4.1.2. Cấu hình Docker Compose

Toàn bộ hệ thống được định nghĩa trong file `docker-compose.yml` tại thư mục gốc của dự án. Cấu hình này đảm bảo:

- **Khởi động có thứ tự (Service Dependencies):** Các service được khởi động theo đúng thứ tự phụ thuộc. PostgreSQL và Redis khởi động trước, sau đó đến Agent Service, rồi Backend, và cuối cùng là Frontend. Cơ chế `depends_on` kết hợp với `healthcheck` đảm bảo service chỉ khởi động khi các service phụ thuộc đã sẵn sàng.
- **Health Checks:** Mỗi service được cấu hình health check riêng để Docker có thể giám sát trạng thái:

- **PostgreSQL:** `pg_isready -U admin -d realestate` – kiểm tra định kỳ mỗi 10 giây, tối đa 5 lần thử.
 - **Redis:** `redis-cli ping` – kiểm tra định kỳ mỗi 10 giây, tối đa 5 lần thử.
 - **Agent Service:** Gọi `/internal/agent/health` với internal key – kiểm tra mỗi 10 giây, tối đa 12 lần thử, có 30 giây khởi động ban đầu.
- **Quản lý dữ liệu bền vững (Persistent Volumes):** Hai Docker volumes được tạo để lưu trữ dữ liệu bền vững:
 - `pgdata`: Lưu toàn bộ dữ liệu PostgreSQL, bao gồm cả dữ liệu quan hệ và vector index.
 - `redisdata`: Lưu dữ liệu cache của Redis.
- Các volume này tồn tại độc lập với vòng đời container, đảm bảo dữ liệu không bị mất khi container được tạo lại.
- **Biến môi trường:** Tất cả cấu hình nhạy cảm được truyền qua biến môi trường từ file `.env`, không được hard-code trong file cấu hình Docker. Các biến có giá trị mặc định an toàn qua cú pháp `${VAR:-default}`.

4.1.3. Cấu hình chi tiết từng service

PostgreSQL + pgvector

```
postgres:
  image: pgvector/pgvector:pg16
  container_name: realestate_postgres
  environment:
    POSTGRES_DB: realestate
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports:
    - "5432:5432"
```

PostgreSQL được triển khai với image `pgvector/pgvector:pg16`, tích hợp sẵn extension `pgvector` cho phép lưu trữ và tìm kiếm vector embedding. Database mặc định được đặt tên là `realestate` với user `admin`. Extension `pgvector` được tự động kích

hoạt thông qua SQLAlchemy trong quá trình khởi tạo schema (hàm `init_db()` trong `backend/app/database.py`).

Redis

```
redis:
  image: redis:7-alpine
  container_name: realestate_redis
  ports:
    - "6379:6379"
  volumes:
    - redisdata:/data
```

Redis sử dụng image Alpine nhẹ (chỉ khoảng 30MB), phù hợp với môi trường container hóa. Redis đảm nhận hai vai trò: caching embedding vector và caching kết quả hybrid search. Dữ liệu cache được lưu bền vững qua volume `redisdata`.

Agent Service

```
agent-service:
  build:
    context: .
    dockerfile: agent_service/Dockerfile
  container_name: realestate_agent_service
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file: .env
  environment:
    DATABASE_URL: postgresql+asyncpg://admin:...@postgres:5432/realestate
    REDIS_URL: redis://redis:6379/0
    AGENT_INTERNAL_KEY: ${AGENT_INTERNAL_KEY}
    GEMINI_API_KEY: ${GEMINI_API_KEY}
    GEMINI_MODEL: gemini-2.0-flash
```

Agent Service được build từ Dockerfile riêng trong thư mục `agent_service/`. Dockerfile dựa trên Python 3.12-slim, cài đặt các thư viện cần thiết (bao gồm LangGraph,

sentence-transformers, httpx) và chạy ứng dụng qua Uvicorn trên port 8100. Biến môi trường PYTHONPATH được thiết lập để Agent Service có thể import các module từ backend/ (models, database).

Điểm đặc biệt trong cấu hình Agent Service là `start_period: 30s` trong health check – khoảng thời gian này cho phép service tải model embedding BGE-M3 (dung lượng khoảng 2GB) trước khi bắt đầu nhận health check request.

Backend API

```
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: realestate_backend
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
    agent-service:
      condition: service_healthy
  env_file: .env
  environment:
    DATABASE_URL: postgresql+asyncpg://admin:...@postgres:5432/realestate
    REDIS_URL: redis://redis:6379/0
    AGENT_SERVICE_URL: http://agent-service:8100
    CHATBOT_AGENT_SERVICE_ENABLED: ${CHATBOT_AGENT_SERVICE_ENABLED:-false}
  volumes:
    - ./data:/app/data
```

Backend API sử dụng Dockerfile trong thư mục backend/, dựa trên Python 3.12-slim với các system dependencies cho asyncpg và bcrypt. Backend chỉ khởi động sau khi cả PostgreSQL, Redis và Agent Service đều đã sẵn sàng (healthy).

Biến `CHATBOT_AGENT_SERVICE_ENABLED` (mặc định: false) cho phép vận hành linh hoạt: khi bật, Backend sẽ chuyển các request chatbot sang Agent Service để xử lý nâng cao; khi tắt, Backend sử dụng pipeline nội bộ. Volume `./data:/app/data` cho phép Backend truy cập dữ liệu CSV và các file cấu hình từ máy host.

Frontend

```
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
    args:
      INTERNAL_API_URL: http://backend:8000
      NEXT_PUBLIC_API_URL: /api/v1
  container_name: realestate_frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
```

Frontend được build từ Dockerfile trong thư mục `frontend/`, sử dụng Node.js 22 Alpine. Quá trình build gồm hai bước: `npm ci` để cài đặt dependencies và `npm run build` để tạo production build. Frontend chạy ở chế độ production (`npm run start`) trên port 3000.

Hai build arguments quan trọng được truyền vào:

- `INTERNAL_API_URL`: URL nội bộ để Next.js server-side gọi Backend API (dùng trong Server Components và SSR).
- `NEXT_PUBLIC_API_URL`: URL công khai để client-side gọi API thông qua browser.

4.1.4. Mạng nội bộ và giao tiếp giữa các service

Các service trong Docker Compose giao tiếp với nhau thông qua tên service (DNS nội bộ của Docker). Cụ thể:

- **Frontend → Backend**: Frontend gọi Backend qua `http://backend:8000` (cho SSR/Server Components) và qua `/api/v1` (cho client-side, được Next.js rewrite).
- **Backend → Agent Service**: Backend gọi Agent Service qua `http://agent-service:8100` với internal key trong header `X-Internal-Agent-Key`.
- **Backend → PostgreSQL**: Backend kết nối đến PostgreSQL qua chuỗi `postgresql+asyncpg`.
- **Backend & Agent Service → Redis**: Cả hai service kết nối đến Redis qua `redis://redis:6379`.

4.1.5. Quy trình triển khai

Chuẩn bị môi trường

Trước khi triển khai, cần chuẩn bị file `.env` với các biến môi trường bắt buộc:

```
# Database
POSTGRES_DB=realestate
POSTGRES_USER=admin
POSTGRES_PASSWORD=<mat_khau_manh>
DATABASE_URL=postgresql+asyncpg://admin:<password>@postgres:5432/realestate

# Redis
REDIS_URL=redis://redis:6379/0

# JWT
JWT_SECRET_KEY=<khoa_bi_mat_jwt>

# CORS
CORS_ORIGINS=http://localhost:3000

# AI/ML
GEMINI_API_KEY=<google_gemini_api_key>
COHERE_API_KEY=<cohere_api_key> # Optional

# Agent Service
AGENT_INTERNAL_KEY=<internal_key>
CHATBOT_AGENT_SERVICE_ENABLED=false # true để sử dụng Agent Service

# Frontend
NEXT_PUBLIC_API_URL=/api/v1
```

Build và khởi động

Quy trình triển khai đầy đủ gồm các bước sau:

1. Build images:

```
docker-compose build
```

Lệnh này build Docker images cho Backend, Agent Service và Frontend từ Dockerfile tương ứng. PostgreSQL và Redis sử dụng images có sẵn từ Docker Hub.

2. Khởi động toàn bộ hệ thống:

```
docker-compose up -d
```

Docker Compose sẽ khởi động các service theo thứ tự: PostgreSQL → Redis → Agent Service → Backend → Frontend. Tham số `-d` chạy ở chế độ nền (detached mode).

3. Khởi động riêng các service hạ tầng (cho phát triển):

```
docker-compose up -d postgres redis
```

Khi phát triển, chỉ cần chạy PostgreSQL và Redis qua Docker, còn Backend và Frontend chạy trực tiếp trên máy host để thuận tiện cho việc debug. Backend: `uvicorn app.main:app --reload --port 8000`. Frontend: `npm run dev`.

4. Khởi động lại một service sau khi thay đổi code:

```
docker-compose up -d --build backend
```

Lệnh này rebuild image và khởi động lại container backend.

5. Xem logs:

```
docker-compose logs -f backend
```

Theo dõi logs của một service cụ thể. Tham số `-f` tương tự `tail -f`.

6. Dừng hệ thống:

```
docker-compose down
```

Dừng và xóa tất cả containers. Dữ liệu trong volumes vẫn được giữ nguyên. Thêm `-v` để xóa cả volumes.

Kiểm tra trạng thái triển khai

Sau khi hệ thống khởi động, có thể kiểm tra trạng thái qua các endpoint health check:

- **Backend API health:** GET `http://localhost:8000/health` – trả về trạng thái database và Redis.
- **Agent Service health:** GET `http://localhost:8100/internal/agent/health` – yêu cầu internal key.
- **Frontend:** Mở trình duyệt tại `http://localhost:3000`.
- **API Documentation:** `http://localhost:8000/docs` – Swagger UI với đầy đủ schema.

4.1.6. Nạp dữ liệu ban đầu

Sau khi hệ thống đã khởi động, cần nạp dữ liệu ban đầu vào PostgreSQL:

```
python -m data_pipeline.load_db
```

Quy trình nạp dữ liệu gồm các bước:

1. Đọc dữ liệu từ các file CSV trong thư mục `data/`.
2. Làm sạch và chuẩn hóa dữ liệu qua `data_pipeline/clean.py`.
3. Làm giàu dữ liệu qua `data_pipeline/enrich.py` (geocoding, intent tagging).
4. Chia văn bản thành chunks qua `data_pipeline/chunk.py`.
5. Tạo embedding BGE-M3 qua `data_pipeline/embed.py`.
6. Upsert vào PostgreSQL qua các ingestor modules.
7. Tạo HNSW index trên cột embedding của bảng chunks.

4.2. Kiểm thử hệ thống

4.2.1. Chiến lược kiểm thử

Hệ thống được kiểm thử theo chiến lược đa tầng (testing pyramid), từ thấp đến cao:

1. **Kiểm thử đơn vị (Unit Testing):** Kiểm tra từng hàm, class, module độc lập. Sử dụng pytest với mock objects để cô lập các phụ thuộc ngoại vi (database, API, file system). Đây là tầng kiểm thử có số lượng test case lớn nhất và chạy nhanh nhất.

2. **Kiểm thử tích hợp (Integration Testing):** Kiểm tra sự tương tác giữa các module với nhau và với các service bên ngoài (PostgreSQL, Redis, Gemini API). Sử dụng test database hoặc mock service.
3. **Kiểm thử hợp đồng (Contract Testing):** Kiểm tra tính tương thích giữa các service qua API contracts (schema request/response). Đảm bảo thay đổi ở một service không phá vỡ service khác.
4. **Kiểm thử hệ thống (System Testing):** Kiểm tra toàn bộ luồng hoạt động end-to-end, từ frontend đến backend đến database. Đảm bảo các chức năng hoạt động đúng như mong đợi của người dùng.
5. **Kiểm thử chấp nhận (Acceptance Testing):** Kiểm tra hệ thống đáp ứng các yêu cầu nghiệp vụ đã đề ra trong pha phân tích. Được thực hiện thủ công với các kịch bản thực tế.

4.2.2. Công cụ và framework kiểm thử

- **pytest:** Framework kiểm thử chính cho toàn bộ code Python (backend, agent_service, data_pipeline, airflow). Sử dụng các plugin: `pytest-asyncio` (hỗ trợ async test), `pytest-cov` (đo coverage), `monkeypatch` (mock built-in).
- **ESLint:** Công cụ phân tích tĩnh (static analysis) cho frontend TypeScript, phát hiện lỗi cú pháp, anti-patterns và vấn đề về coding style.
- **compileall:** Module chuẩn của Python dùng để kiểm tra cú pháp toàn bộ file `.py` trong dự án, phát hiện lỗi import và syntax error mà không cần chạy test.
- **Thư viện mock/asyncio:** Sử dụng `unittest.mock` và `pytest.monkeypatch` để tạo mock objects, giả lập các service bên ngoài (database, API, LLM).

KẾT LUẬN

Kết quả đạt được

Sau quá trình nghiên cứu và triển khai, đề án đã đạt được các kết quả chính sau:

1. **Xây dựng thành công nền tảng web bất động sản hoàn chỉnh** với frontend Next.js 16 + React 19 và backend FastAPI, bao gồm các chức năng: duyệt danh sách nhà đất bán/cho thuê, lọc đa tiêu chí, sắp xếp, xem chi tiết bất động sản, dashboard thống kê thị trường, hệ thống xác thực người dùng qua JWT, và bảng điều khiển quản trị (admin dashboard).
2. **Phát triển chatbot đa tác tử (Multi-Agent) hoạt động ổn định** với 4 agent chuyên biệt (Property Search, Market Analysis, Legal Advisor, Investment Advisor) và router phân loại ý định. Chatbot có khả năng hiểu ý định người dùng bằng tiếng Việt qua cả keyword-based routing và Gemini-based routing, điều phối đến agent phù hợp, truy xuất dữ liệu qua hybrid search (SQL + pgvector kNN + Cohere rerank + Redis cache) và sinh phản hồi có trích dẫn nguồn rõ ràng.
3. **Triển khai kiến trúc chatbot hai lớp** với pipeline nội bộ (backend/app/services/chatbot/) xử lý nhanh cho người dùng cuối và Agent Service (agent_service/) chạy LangGraph StateGraph riêng biệt cho multi-step reasoning nâng cao. Hệ thống tích hợp cơ chế memory và preferences để ghi nhớ ngữ cảnh người dùng.
4. **Xây dựng pipeline thu thập và xử lý dữ liệu tự động** từ batdongsan.com.vn, bao gồm: crawl dữ liệu bằng Playwright với stealth mode cho cả tin bán, tin cho thuê, dự án và tin tức; làm sạch và chuẩn hóa; làm giàu (geocoding qua Nominatim/Goong, trích xuất intent tags); chunking ngữ nghĩa; tạo embedding BGE-M3 (1024 chiều); và nạp vào PostgreSQL + pgvector qua các ingestor module. Pipeline được lập lịch tự động qua Apache Airflow với 4 DAGs.
5. **Triển khai hệ thống tìm kiếm lai hiệu quả** kết hợp giữa SQL filtering cho dữ liệu có cấu trúc và pgvector kNN với HNSW index cho tìm kiếm ngữ nghĩa, có reranking qua Cohere và caching qua Redis để cải thiện độ chính xác và tốc độ.
6. **Thiết kế kiến trúc microservices có khả năng mở rộng** với Docker Compose, tách biệt 5 service (frontend, backend, agent service, database, cache), hỗ trợ triển khai dễ dàng trên nhiều môi trường.

7. **Xây dựng hệ thống observability toàn diện** với agent tracing (AgentTrace, AgentTraceStep, AgentLLMCall, AgentRetrievalEvent), chat feedback, memory proposals, source readiness monitoring và Prometheus metrics endpoint.
8. **Bộ test phong phú** với hơn 60 file test bao phủ toàn bộ các module: chatbot pipeline, agent service, hybrid search, RAG, data pipeline, legal KB, crawler, memory/preferences và admin.

Hạn chế

Bên cạnh những kết quả đạt được, đồ án vẫn còn một số hạn chế:

PHỤ LỤC

Phụ lục A CẤU TRÚC THƯ MỤC DỰ ÁN

Cấu trúc thư mục chính của dự án (rút gọn):

```
1 RealEstate_Chatbot_v2/|—
2   frontend/                                     # Next.js 16 + React 19 + TypeScript|—
3     app/|||—
4       page.tsx                                   # Trang ùch||—
5       layout.tsx                                # Root layout||—
6       nha-dat-ban/                               # Danh sách & chi ếtit ĐBS bán|||—
7         page.tsx|||—
8         [id]/                                   # Trang chi ếtit (dynamic route)||—
9       nha-dat-cho-thue/                         # Danh sách & chi ếtit cho thuê||—
10      thi-truong/                               # Dashboard ồthng kê ịth uờtrng||—
11      admin/                                    # àBng ềđiêu ểkhin àqun ịtr||—
12      dang-nhap/                                # Đăng ậnhp||—
13      dang-ky/                                  # Đăng ký|—
14      components/|||—
15        layout/                                 # Header.tsx, Footer.tsx||—
16        listing/                               # ListingCard.tsx, ListingGrid.tsx||—
17        search/                                # FilterPanel.tsx||—
18        chatbot/                               # ChatWidget.tsx||—
19        admin/                                 # AdminDashboard.tsx|—
20      lib/|||—
21        api.ts                                  # Typed API client (FastAPI)||—
22        types.ts                               # TypeScript interfaces||—
23        utils.ts                               # Format helpers||—
24        chatSourceDisplay.ts                   # Chat source rendering|—
25      public/                                  # Static assets|—
26
27      backend/                                  # Backend API (FastAPI + SQLAlchemy)|—
28        app/|||—
29          main.py                               # FastAPI app, CORS, router registration||—
30          config.py                             # Settings (env vars, defaults)||—
31          database.py                           # Async engine, session, Base||—
32          models/                               # 14+ ORM models|||—
33            listing.py                          # - listings (30+ fields)|||—
34            chunk.py                            # - chunks (pgvector 1024d)|||—
35            chat.py                             # - chat_sessions, chat_messages|||—
36            user.py                             # - users|||—
37            project.py                          # - projects|||—
38            article.py                          # - articles|||—
39            pipeline_run.py                     # - pipeline_runs|||—
```

```

40     preference.py      #   - user_preferences,|||
41                       #   memory_proposals,|||
42                       #   chat_feedback|||└─
43     agent_observability.py # agent_traces,|||
44                       #   agent_trace_steps,|||
45                       #   agent_llm_calls,|||
46                       #   agent_retrieval_events|||└─
47     source_readiness.py  # source_readiness||└─
48     schemas/             # Pydantic v2 contracts|||└─
49     listing.py, chat.py, auth.py|||└─
50     common.py, admin.py, preferences.py||└─
51     routers/            # 7 router modules|||└─
52     listings.py         #   - /api/v1/listings|||└─
53     market.py          #   - /api/v1/market|||└─
54     chat.py             #   - /api/v1/chat|||└─
55     auth.py             #   - /api/v1/auth|||└─
56     preferences.py      #   - /api/v1/preferences|||└─
57     admin.py            #   - /api/v1/admin||└─
58     metrics.py          #   - /metrics||└─
59     services/||└─
60     chatbot/            # Pipeline multi-agent noi bo|||└─
61     router.py           #   Intent routing (keyword + Gemini)|||└─
62     orchestrator.py     #   Agent coordination|||└─
63     context.py          #   Conversation context|||└─
64     memory.py           #   User memory & preferences|||└─
65     contracts.py        #   RoutingDecision, AgentResult|||└─
66     analytics.py        #   Pipeline analytics||└─
67     agents/            #   4 specialist agents|||└─
68     property.py, market.py|||└─
69     legal.py, investment.py||└─
70     rag/               # RAG services|||└─
71     hybrid_search.py    #   SQL + pgvector + rerank + cache|||└─
72     embeddings.py       #   BGE-M3 embedder|||└─
73     simple_rag.py       #   Filter extraction|||└─
74     cache.py            #   Redis caching||└─
75     ingest.py           #   Vector store ingestion||└─
76     agent_service/      # Agent Service client||└─
77     client.py           #   HTTP client + internal key auth||└─
78     contracts.py        #   Request/response schemas|└─
79     tests/              # 58 test files||└─
80     conftest.py         #   Shared fixtures||└─
81     fixtures/           #   Test data||└─
82     test_*.py           #   Organized by module|└─
83     alembic/            #   DB migrations|└─
84     scripts/            #   Utility scripts|└─
85     requirements.txt|└─
86     Dockerfile|└─

```



```

87
88 agent_service/                                # Internal LangGraph Agent Service| |—
89     main.py                                    # FastAPI app (port 8100)| |—
90     config.py                                 # Agent-specific settings| |—
91     security.py                              # Internal key authentication| |—
92     contracts.py                             # AgentChatRequest/Response| |—
93     graph/| | |—
94         workflow.py                           # 8-node StateGraph| | |—
95         state.py                              # AgentGraphState| | |—
96         nodes.py                             # Node implementations| | |—
97         retrieval_planner.py                 # Multi-step retrieval planning| |—
98     agents/| | |—
99         specialists.py                       # 4 specialist agents| |—
100    tools/| | |—
101        retrieval.py                          # Hybrid search tool| | |—
102        market.py                            # Market stats tool| | |—
103        readiness.py                         # Data readiness check| |—
104    llm/| | |—
105        gemini.py                             # Gemini 2.0 Flash wrapper| |—
106    evaluation/| | |—
107        judge.py                             # LLM Judge (5 metrics)| |—
108    requirements.txt| |—
109    Dockerfile| |—
110
111    crawler/                                    # Playwright web scrapers| |—
112        core/                                # Shared infrastructure| | |—
113            csv_writer.py                    # CSV with dedup| | |—
114            parser.py                        # Base parser| | |—
115            listing_detail_parser.py         # Full detail parser| |—
116    sale/                                      # Sale listings| | |—
117        crawl_urls.py                        # URL discovery (30 pages)| | |—
118        crawl_details.py                    # Detail scraping (4 workers)| |—
119    rent/                                    # Rental listings| | |—
120        crawl_urls.py| | |—
121        crawl_details.py| |—
122    projects/                                # Real estate projects| | |—
123        crawl_urls.py| | |—
124        crawl_details.py| |—
125    news/                                    # News & articles| |—
126        crawl_articles.py| |—
127
128    data_pipeline/                            # ETL pipeline| |—
129        clean.py                             # Data cleaning & normalization| |—
130        enrich.py                            # Geocoding + intent tagging| |—
131        chunk.py                             # Semantic chunking (4 types)| |—
132        embed.py                             # BGE-M3 embedding (1024d, batch 16)| |—
133        load_db.py                           # PostgreSQL upsert| |—

```

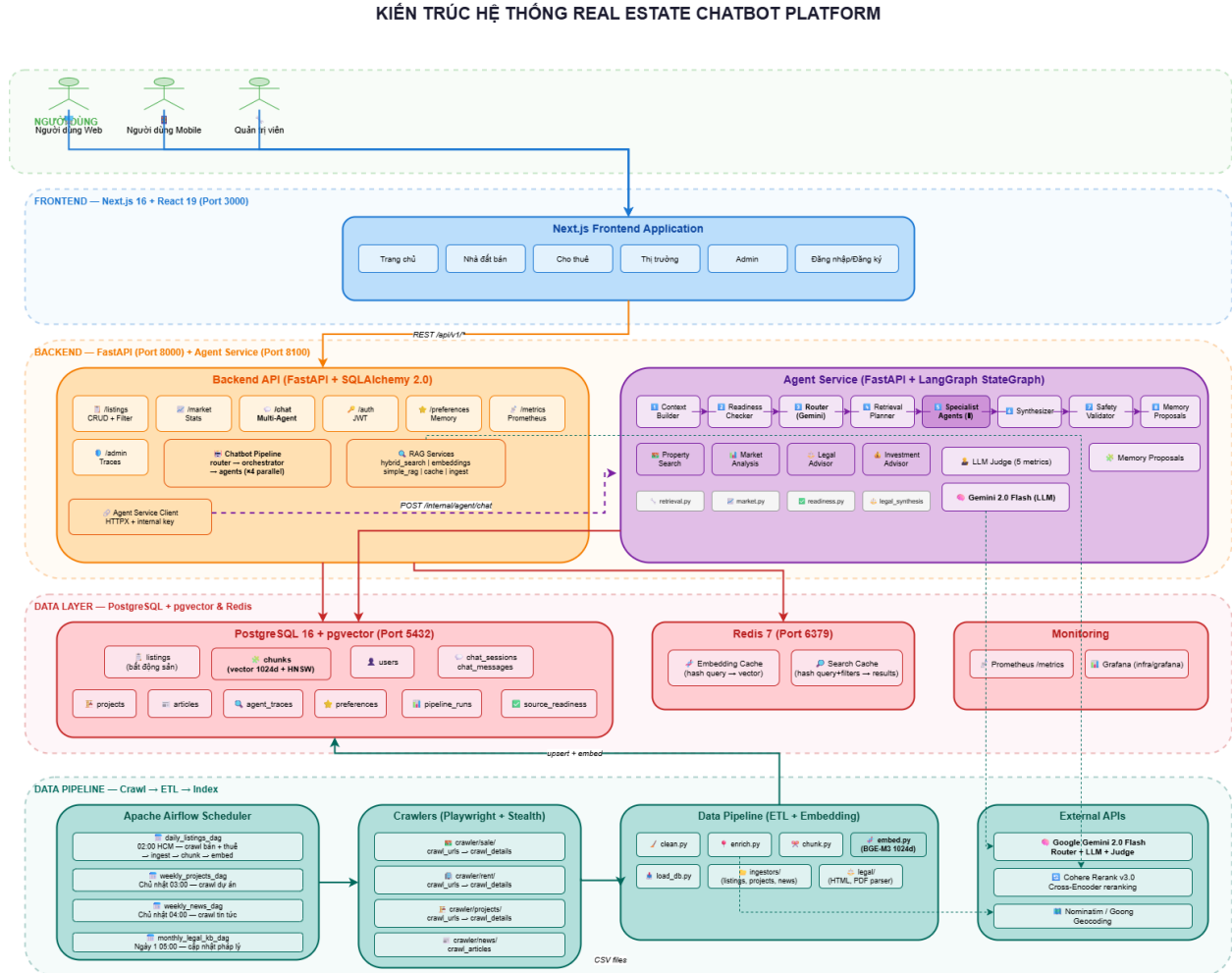
```

134 ingestors/ # Source-specific ingestors||└─
135     listings_ingestor.py||└─
136     projects_ingestor.py||└─
137     news_ingestor.py||└─
138     legal_kb_ingestor.py|└─
139 legal/ # Legal KB processing|└─
140     chunker.py # Legal document chunking|└─
141     html_parser.py # HTML legal docs|└─
142     pdf_parser.py # PDF legal docs|└─
143     structure.py # Document structure|└─
144     manifest.py # Version tracking|└─
145     slug_disambiguation.py|└─
146
147 airflow/ # Workflow orchestration|└─
148     dags/ # 4 scheduled DAGs||└─
149         daily_listings_dag.py # Daily 02:00 HCM||└─
150         weekly_projects_dag.py # Sunday 03:00||└─
151         weekly_news_dag.py # Sunday 04:00||└─
152         monthly_legal_kb_dag.py # 1st day 05:00|└─
153     plugins/ # Airflow plugins||└─
154         pipeline_runner.py # Crawl & ingest runner||└─
155         alerting.py # Slack + email alerts||└─
156         run_metrics.py # Pipeline metrics|└─
157     Dockerfile|└─
158     requirements.txt|└─
159
160 chatbot/ # Legacy LangGraph sandbox|└─
161     graph.py, state.py, config.py|└─
162     agents/ # Experimental agents|└─
163     tools/ # Experimental tools|└─
164
165 docs/ # Documentation|└─
166     architecture.drawio # System architecture diagram|└─
167     implementation_plan.md|└─
168     multiagent-workflow.md|└─
169     pipeline.md|└─
170
171 report/ # LaTeX report|└─
172     main.tex|└─
173     preamble.tex|└─
174     contents/ # Chapter files|└─
175     figures/ # Architecture diagram (PNG)|└─
176
177 data/ # Data directory|└─
178     raw/ # Raw crawl output (CSV)|└─
179     knowledge/ # Legal KB source files|└─

```


Phụ lục B SƠ ĐỒ KIẾN TRÚC & LUỒNG DỮ LIỆU

Sơ đồ kiến trúc tổng thể



Hình 4.2: Sơ đồ kiến trúc tổng thể hệ thống (5 tầng: Người dùng – Frontend – Backend & Agent Service – Data – Pipeline)

Luồng dữ liệu từ Crawl đến Hiển thị

- Crawl (Playwright + Stealth):** Thu thập dữ liệu từ batdongsan.com.vn với 4 workers song song. Output: file CSV UTF-8 BOM, deduplicate theo product_id. Thực hiện bởi 4 module crawler (sale, rent, projects, news).
- Clean (data_pipeline/clean.py):** Chuẩn hóa dữ liệu thô – parse giá (“4,68 tỷ” → price=4.68, unit=billion), parse diện tích (“75 m²” → area=75), phân loại property_type, chuẩn hóa địa chỉ (tách ward, district, city).
- Enrich (data_pipeline/enrich.py):** Làm giàu dữ liệu với 2 tác vụ: (a) Geocoding –

gọi Nominatim/Goong API chuyển địa chỉ text → tọa độ (latitude, longitude); (b) Intent Tagging – phân tích mô tả, gán nhãn nhu cầu (“gần trường học”, “view đẹp”, “an ninh”, “pháp lý rõ”).

4. **Chunk (data_pipeline/chunk.py):** Chia mỗi listing thành 4 loại chunk ngữ nghĩa độc lập:
 - `overview` – Tổng quan: tiêu đề, giá, diện tích, khu vực, pháp lý, nội thất.
 - `description` – Mô tả chi tiết từ người đăng.
 - `location` – Vị trí: địa chỉ, quận/huyện, tỉnh/thành phố.
 - `intent_tags` – Nhu cầu phù hợp (dựa trên intent tagging).
5. **Embed (data_pipeline/embed.py):** Tạo vector embedding 1024 chiều cho từng chunk bằng BGE-M3 (BAAI/bge-m3). Sử dụng sentence-transformers với `batch_size=16`, `normalize=True`, retry 3 lần exponential backoff.
6. **Load DB (data_pipeline/load_db.py + ingestors/):** Upsert vào PostgreSQL qua asyncpg driver với batch processing (25–50 bản ghi/batch). Dữ liệu quan hệ → bảng listings; dữ liệu vector → bảng chunks với HNSW index (`m=16`, `ef_construction=64`).
7. **Serve (FastAPI + Next.js):** Backend API truy xuất dữ liệu qua hybrid search (SQL filtering + pgvector kNN + Cohere rerank + Redis cache). Frontend hiển thị qua Next.js App Router với SSR.
8. **Schedule (Apache Airflow):** Toàn bộ pipeline từ bước 1–6 được lập lịch tự động qua 4 DAGs (daily, weekly×2, monthly) với retry policy và alerting.

Luồng xử lý Chatbot

1. **User gửi query** qua ChatWidget (Frontend).
2. **Frontend gọi** `POST /api/v1/chat` (Backend).
3. **Backend xác thực** (JWT hoặc anonymous), kiểm tra quota, lưu ChatMessage.
4. **Router phân loại ý định** (keyword-based hoặc Gemini-based).
5. **Orchestrator chọn agents** và gọi song song qua `asyncio.gather()`.
6. **Agent thực thi** hybrid search (SQL + pgvector kNN ± Cohere rerank ± Redis cache).
7. **Kết quả tổng hợp** → AgentChatResponse (`final_response`, `sources`, `suggested_actions`).

8. **Nếu Agent Service được kích hoạt**, Backend ủy thác sang Agent Service (LangGraph 8-node workflow) thay vì pipeline nội bộ.
9. **Backend lưu** ChatMessage, AgentTrace và trả kết quả về Frontend.
10. **Frontend hiển thị** phản hồi kèm sources và suggested actions.

Phụ lục C DANH SÁCH API CHÍNH

Bảng 4.1 liệt kê các API endpoint chính của hệ thống (33 endpoints, 7 routers + metrics). Tất cả endpoint đều có tiền tố /api/v1 trừ /metrics và /api/v1/health.

Endpoint	Method	Mô tả
Listings		
GET /listings	GET	Danh sách BĐS (phân trang, 12+ bộ lọc: search, type, city, district, price, area, bedrooms,...)
GET /listings/{id}	GET	Chi tiết một BĐS (30+ trường)
GET /listings/by-product-id/{pid}	GET	Tìm BĐS theo product_id gốc từ nguồn crawl
GET /listings/similar/{id}	GET	BDS tương tự (cùng quận, loại hình, phân khúc giá)
Market		
GET /market/stats	GET	Tổng quan thị trường (tổng tin, giá TB, diện tích TB, số TP/quận)
GET /market/top-locations	GET	Top khu vực nhiều tin nhất (lọc theo listing_type)
GET /market/price-by-district	GET	Thống kê giá theo quận (avg, min, max, giá/m²)
GET /market/property-types	GET	Phân bố số lượng theo loại hình BĐS
GET /market/cities	GET	Danh sách thành phố + số lượng tin
GET /market/districts	GET	Danh sách quận/huyện (có thể lọc theo city)
GET /market/categories	GET	Danh sách tất cả loại hình BĐS
Chat		
POST /chat	POST	Gửi tin nhắn đến chatbot (tự động tạo session nếu chưa có)
GET /chat/sessions	GET	Danh sách phiên chat của user (cần xác thực)
GET /chat/sessions/{id}	GET	Lịch sử tin nhắn của một phiên chat
POST /chat/feedback	POST	Gửi đánh giá (rating, issue_type, comment)
Auth		
POST /auth/register	POST	Đăng ký tài khoản (email, password, full_name)
POST /auth/login	POST	Đăng nhập, nhận JWT token (hết hạn sau 24h)
GET /auth/me	GET	Thông tin người dùng hiện tại (cần xác thực)
Preferences & Memory		
GET/PATCH /preferences	GET/PATCH	Xem/cập nhật sở thích người dùng (cần xác thực)
POST /memory-proposals/{id}/accept	POST	Chấp nhận đề xuất ghi nhớ của chatbot
POST /memory-proposals/{id}/reject	POST	Từ chối đề xuất ghi nhớ
Admin (cần quyền admin)		
GET /admin/chat-traces	GET	Danh sách agent traces (intent, agents, latency)
GET /admin/chat-traces/{id}	GET	Chi tiết một trace (từng bước, input/output)
GET /admin/pipeline-readiness	GET	Trạng thái sẵn sàng của các nguồn dữ liệu

GET /admin/eval-runs	GET	Danh sách kết quả đánh giá LLM Judge
GET /admin/feedback	GET	Danh sách phản hồi người dùng về chatbot
GET /admin/memory-proposals	GET	Danh sách memory proposals đang chờ xử lý
GET /admin/agent-health	GET	Thống kê sức khỏe agent theo trạng thái
GET /admin/top-queries	GET	Top truy vấn phổ biến (đang phát triển)
System		
GET /api/v1/health	GET	Health check endpoint
GET /metrics	GET	Prometheus metrics (chat requests, latency, pipeline)

Bảng 4.1: Danh sách API endpoints chính của hệ thống (tiền tố mặc định: /api/v1)

Phụ lục D VÍ DỤ PHẢN HỒI CHATBOT

Ví dụ 1: Tìm kiếm bất động sản

User: “Tôi muốn mua căn hộ chung cư ở Quận 7, giá dưới 5 tỷ, 2 phòng ngủ”

Intent: property_search

Agent sử dụng: property_search

Bộ lọc trích xuất: {listing_type: "sale", property_type: "apartment", district: "Quận 7", max_price: 5, bedrooms: 2}

Phản hồi: Dựa trên yêu cầu của bạn, tôi tìm thấy một số căn hộ phù hợp tại Quận 7:

1. **Căn hộ chung cư ABC** – 4,2 tỷ, 72m², 2PN, 2WC, Phường Tân Phong. View sông, nội thất đầy đủ, pháp lý sổ hồng.
2. **Căn hộ Sunshine City** – 4,8 tỷ, 78m², 2PN, 1WC, Phường Phú Thuận. Gần trường học, an ninh tốt, có hồ bơi.
3. **Căn hộ Park View** – 3,9 tỷ, 68m², 2PN, 1WC, Phường Tân Quy. Giá tốt nhất khu vực, gần siêu thị.

Nguồn tham khảo: [Link BĐS 1] (độ liên quan: 0.95), [Link BĐS 2] (0.89), [Link BĐS 3] (0.82)

Gợi ý tiếp theo:

- “So sánh giá/m² các căn hộ này”
- “Thủ tục pháp lý khi mua chung cư cần những gì?”
- “Khu vực Quận 7 có tiềm năng tăng giá không?”

Ví dụ 2: Tư vấn pháp lý + thị trường

User: “Thủ tục sang tên sổ đỏ mất bao lâu? Với lại giá đất ở Quận 9 hiện nay thế nào?”

Intent: mixed (legal_advice + market_analysis)

Agent sử dụng: legal_advisor, market_analysis

Phản hồi: Tôi sẽ giải đáp từng phần câu hỏi của bạn:

1. Thủ tục sang tên sổ đỏ:

Theo quy định hiện hành, thủ tục sang tên sổ đỏ (chuyển nhượng quyền sử dụng đất) gồm các bước chính:

- Công chứng hợp đồng chuyển nhượng tại văn phòng công chứng (1–2 ngày).
- Khai thuế thu nhập cá nhân và lệ phí trước bạ tại Chi cục Thuế (3–5 ngày).
- Nộp hồ sơ sang tên tại Văn phòng Đăng ký Đất đai (10–15 ngày làm việc).

Tổng thời gian ước tính: **3–5 tuần** tùy địa phương.

Lưu ý quan trọng: Cần kiểm tra đất không có tranh chấp, không bị quy hoạch, và đã hoàn thành nghĩa vụ tài chính.

2. Giá đất tại Quận 9:

Hiện tại (dữ liệu cập nhật từ hệ thống):

- Giá đất nền trung bình: **35–55 triệu/m²** tùy vị trí.
- Giá đất mặt tiền đường lớn: **60–90 triệu/m²**.
- Xu hướng: tăng nhẹ 5–8% so với cùng kỳ năm trước, được thúc đẩy bởi hạ tầng giao thông (Metro, Vành đai 3).

Nguồn tham khảo: [Link KB Pháp lý 1], [Link KB Pháp lý 2], [Link Thống kê Quận 9]

Gợi ý tiếp theo:

- “Các loại thuế phí khi mua đất là gì?”
- “So sánh giá đất Quận 9 với Quận 2 và Thủ Đức”
- “Có nên đầu tư đất nền Quận 9 thời điểm này?”

CHỈ MỤC

TÀI LIỆU THAM KHẢO